

UNIVERSITAT POLITÈCNICA DE VALÈNCIA  
ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA INFORMÁTICA

Grado en Ciencia de Datos

---

# Adaptación del modelo OWSM para la decodificación de actividad neuronal intracortical en el reto Brain-to-Text '25

---

Trabajo de Fin de Grado

**Autor:** Kiril Ivaylov Tzenkov

**Tutor:** Nombre del Tutor

**Cotutor:** Nombre del Cotutor

Curso 2025–2026

# Resumen

Las interfaces cerebro-computador (BCI) para el habla buscan restaurar la comunicación en personas con pérdida de la capacidad de hablar. Este trabajo estudia la viabilidad de adaptar OWSM, un modelo de reconocimiento de voz preentrenado de propósito general, a la decodificación de actividad neuronal intracortical del reto *Brain-to-Text* '25. Mediante la eliminación del *frontend* acústico, la incorporación de un módulo adaptador del espacio de características neuronales y una serie de ablaciones controladas sobre la estrategia de adaptación, se evalúa en qué medida un modelo de habla puede transferirse a un dominio para el que no fue concebido.

**Palabras clave:** interfaz cerebro-computador, actividad neuronal intracortical, decodificación neuronal, OWSM, transferencia de aprendizaje.

# Abstract

Speech brain–computer interfaces (BCIs) aim to restore communication for people who have lost the ability to speak. This work investigates the feasibility of adapting OWSM, a general-purpose pretrained speech recognition model, to decode intracortical neural activity from the *Brain-to-Text* '25 challenge. By removing the acoustic frontend, introducing an adapter module for the neural feature space, and running a series of controlled ablations over the adaptation strategy, we assess to what extent a speech model can be transferred to a domain it was not designed for.

**Keywords:** brain–computer interface, intracortical neural activity, neural decoding, OWSM, transfer learning.

# Índice general

|                                                                              |           |
|------------------------------------------------------------------------------|-----------|
| <b>Resumen</b>                                                               | <b>I</b>  |
| <b>Abstract</b>                                                              | <b>II</b> |
| <b>1. Introducción</b>                                                       | <b>1</b>  |
| 1.1. Contexto . . . . .                                                      | 1         |
| 1.2. Del control motor al habla: los sistemas <i>brain-to-text</i> . . . . . | 2         |
| 1.3. Planteamiento del trabajo . . . . .                                     | 3         |
| 1.4. Motivación . . . . .                                                    | 4         |
| 1.4.1. Motivación clínica y social . . . . .                                 | 4         |
| 1.4.2. Motivación científica y técnica . . . . .                             | 4         |
| 1.4.3. Motivación de accesibilidad y reproducibilidad . . . . .              | 5         |
| 1.4.4. Motivación personal . . . . .                                         | 5         |
| 1.5. Objetivos . . . . .                                                     | 5         |
| 1.5.1. Objetivo general . . . . .                                            | 5         |
| 1.5.2. Preguntas de investigación . . . . .                                  | 6         |
| 1.5.3. Objetivos específicos . . . . .                                       | 6         |
| 1.5.4. Alcance y limitaciones . . . . .                                      | 8         |
| 1.6. Estructura de la memoria . . . . .                                      | 9         |
| <b>2. Estado del arte</b>                                                    | <b>10</b> |
| 2.1. Modalidades de registro en interfaces cerebro-computador . . . . .      | 10        |
| 2.2. Evolución de las neuroprótesis del habla . . . . .                      | 11        |
| 2.3. Anatomía de un decodificador <i>brain-to-text</i> . . . . .             | 12        |
| 2.3.1. Características de entrada y no estacionariedad . . . . .             | 12        |

|                                                                        |           |
|------------------------------------------------------------------------|-----------|
| 2.3.2. El codificador neuronal . . . . .                               | 13        |
| 2.3.3. Objetivo de entrenamiento y unidad lingüística . . . . .        | 13        |
| 2.3.4. El modelo de lenguaje y la formación de palabras . . . . .      | 14        |
| 2.4. Los <i>benchmarks</i> Brain-to-Text . . . . .                     | 14        |
| 2.5. Modelos de habla preentrenados . . . . .                          | 15        |
| 2.6. Adaptación eficiente en parámetros . . . . .                      | 16        |
| 2.7. Transferencia de modelos de habla a señal neuronal . . . . .      | 16        |
| 2.8. Posicionamiento de este trabajo . . . . .                         | 17        |
| <b>3. Conjunto de datos</b>                                            | <b>19</b> |
| 3.1. Criterios de selección . . . . .                                  | 19        |
| 3.2. Alternativas evaluadas . . . . .                                  | 20        |
| 3.2.1. UGR-MINDVOICE . . . . .                                         | 20        |
| 3.2.2. sEEG2Speech . . . . .                                           | 21        |
| 3.2.3. SingleWordProductionDutch . . . . .                             | 21        |
| 3.2.4. Brain-to-Text '25 . . . . .                                     | 22        |
| 3.3. El conjunto Brain-to-Text '25 . . . . .                           | 22        |
| 3.3.1. Origen y participante . . . . .                                 | 22        |
| 3.3.2. Adquisición y características neuronales . . . . .              | 22        |
| 3.3.3. Organización de los datos . . . . .                             | 23        |
| 3.3.4. Particiones y composición por corpus . . . . .                  | 24        |
| 3.3.5. Estrategias de habla no etiquetadas . . . . .                   | 24        |
| 3.4. Análisis exploratorio . . . . .                                   | 25        |
| 3.4.1. Distribución de longitudes . . . . .                            | 25        |
| 3.4.2. Variabilidad entre sesiones . . . . .                           | 25        |
| 3.4.3. Cobertura léxica y ejemplos por clase . . . . .                 | 25        |
| 3.4.4. Distribución temporal del registro . . . . .                    | 25        |
| 3.5. Retos que plantea el conjunto y consecuencias de diseño . . . . . | 25        |
| <b>4. Marco teórico</b>                                                | <b>27</b> |
| 4.1. Planteamiento formal del problema . . . . .                       | 27        |
| 4.2. Modelado de secuencias con codificadores atencionales . . . . .   | 28        |

|           |                                                                     |           |
|-----------|---------------------------------------------------------------------|-----------|
| 4.2.1.    | Autoatención multicabeza . . . . .                                  | 28        |
| 4.2.2.    | El compromiso entre contexto local y global en señales temporales   | 28        |
| 4.2.3.    | E-Branchformer . . . . .                                            | 29        |
| 4.3.      | OWSM v3.1 . . . . .                                                 | 29        |
| 4.3.1.    | Origen y motivación . . . . .                                       | 29        |
| 4.3.2.    | Arquitectura . . . . .                                              | 30        |
| 4.3.3.    | Objetivo conjunto CTC-atención . . . . .                            | 30        |
| 4.4.      | Clasificación temporal conexionista (CTC) . . . . .                 | 31        |
| 4.4.1.    | Formulación . . . . .                                               | 31        |
| 4.4.2.    | Decodificación voraz . . . . .                                      | 31        |
| 4.4.3.    | Regularización intermedia (InterCTC) . . . . .                      | 32        |
| 4.4.4.    | Propiedades y limitaciones . . . . .                                | 32        |
| 4.5.      | Adaptación eficiente en parámetros: LoRA . . . . .                  | 32        |
| 4.6.      | Modelado de lenguaje y decodificación . . . . .                     | 33        |
| 4.6.1.    | Modelos de lenguaje de $n$ -gramas . . . . .                        | 33        |
| 4.6.2.    | Búsqueda en haz sobre salidas CTC . . . . .                         | 34        |
| 4.6.3.    | Fusión superficial . . . . .                                        | 34        |
| 4.6.4.    | Guiado frente a reordenamiento, y el límite <i>oracle</i> . . . . . | 34        |
| 4.7.      | Métricas de evaluación . . . . .                                    | 35        |
| 4.7.1.    | Distancia de edición . . . . .                                      | 35        |
| 4.7.2.    | CER, PER y WER . . . . .                                            | 35        |
| 4.7.3.    | Comparabilidad entre métricas . . . . .                             | 35        |
| <b>5.</b> | <b>Arquitectura del sistema</b>                                     | <b>37</b> |
| 5.1.      | Visión general . . . . .                                            | 37        |
| 5.2.      | Adaptación de modalidad . . . . .                                   | 38        |
| 5.2.1.    | Qué se conserva y qué se sustituye . . . . .                        | 38        |
| 5.2.2.    | Desactivación del decodificador . . . . .                           | 39        |
| 5.3.      | Capa específica de sesión . . . . .                                 | 39        |
| 5.3.1.    | Motivación y formulación . . . . .                                  | 39        |
| 5.3.2.    | Transporte del índice de sesión . . . . .                           | 40        |

|                                                                 |           |
|-----------------------------------------------------------------|-----------|
| 5.4. Módulo adaptador . . . . .                                 | 41        |
| 5.4.1. Variante convolucional bidimensional . . . . .           | 41        |
| 5.4.2. Variante lineal . . . . .                                | 41        |
| 5.4.3. Variante convolucional unidimensional . . . . .          | 41        |
| 5.4.4. Variante profunda . . . . .                              | 41        |
| 5.4.5. Anulación del relleno . . . . .                          | 42        |
| 5.5. Cabeza CTC y unidades de salida . . . . .                  | 43        |
| 5.6. Estrategias de adaptación del codificador . . . . .        | 43        |
| 5.6.1. Interacción entre LoRA y la congelación . . . . .        | 44        |
| 5.6.2. Tasa de aprendizaje diferencial . . . . .                | 44        |
| 5.6.3. Codificador recortado . . . . .                          | 44        |
| 5.6.4. El control de inicialización . . . . .                   | 45        |
| 5.7. Preprocesado y aumento de datos . . . . .                  | 45        |
| 5.7.1. Normalización . . . . .                                  | 45        |
| 5.7.2. Aumento de datos . . . . .                               | 45        |
| 5.7.3. Suavizado temporal . . . . .                             | 46        |
| 5.8. Recuento de parámetros . . . . .                           | 46        |
| 5.9. Decisiones de implementación . . . . .                     | 46        |
| <b>6. Metodología experimental</b>                              | <b>48</b> |
| 6.1. Entorno de ejecución y presupuesto de cómputo . . . . .    | 48        |
| 6.2. Preparación de los datos . . . . .                         | 49        |
| 6.3. Receta de entrenamiento . . . . .                          | 49        |
| 6.4. Selección de modelo y parada temprana . . . . .            | 51        |
| 6.5. Protocolo de evaluación . . . . .                          | 51        |
| 6.5.1. Dos regímenes de decodificación . . . . .                | 51        |
| 6.5.2. Dos definiciones de tasa de error por carácter . . . . . | 52        |
| 6.5.3. Precisión de la medida . . . . .                         | 53        |
| 6.5.4. Comprobaciones de sanidad . . . . .                      | 53        |
| 6.6. Principios metodológicos . . . . .                         | 53        |
| 6.6.1. Una variable por experimento . . . . .                   | 53        |

|                                                                   |           |
|-------------------------------------------------------------------|-----------|
| 6.6.2. Presupuesto de cómputo igualado . . . . .                  | 54        |
| 6.6.3. Trazabilidad . . . . .                                     | 54        |
| 6.7. Diseño experimental . . . . .                                | 54        |
| 6.7.1. Bloque A: el control científico . . . . .                  | 55        |
| 6.7.2. Bloque D: la curva de conservación . . . . .               | 55        |
| 6.7.3. Bloque H: blindaje del hallazgo . . . . .                  | 56        |
| 6.7.4. Bloque F: el tamaño de la arquitectura . . . . .           | 56        |
| 6.7.5. Bloque B: granularidad de salida . . . . .                 | 56        |
| 6.7.6. Bloque E: generalización a días no vistos . . . . .        | 56        |
| 6.7.7. Bloque C: capacidad del adaptador . . . . .                | 56        |
| 6.7.8. Bloque G: configuración final . . . . .                    | 57        |
| 6.8. Protocolo de decodificación con modelo de lenguaje . . . . . | 57        |
| 6.8.1. Construcción del modelo de lenguaje . . . . .              | 57        |
| 6.8.2. Ajuste de los hiperparámetros de fusión . . . . .          | 57        |
| 6.8.3. Cifras reportadas . . . . .                                | 57        |
| <b>7. Resultados y análisis</b>                                   | <b>59</b> |
| 7.1. Resultados cuantitativos . . . . .                           | 59        |
| 7.2. Análisis de errores . . . . .                                | 59        |
| 7.3. Comportamiento por sesión . . . . .                          | 59        |
| 7.4. Curvas de entrenamiento . . . . .                            | 59        |
| 7.5. Discusión . . . . .                                          | 59        |
| <b>8. Limitaciones y trabajo futuro</b>                           | <b>60</b> |
| 8.1. Limitaciones computacionales . . . . .                       | 60        |
| 8.2. Limitaciones del enfoque . . . . .                           | 60        |
| 8.3. Líneas de trabajo futuro . . . . .                           | 60        |
| <b>9. Conclusiones</b>                                            | <b>61</b> |
| <b>Bibliografía</b>                                               | <b>62</b> |
| <b>A. Código del pipeline de entrenamiento</b>                    | <b>63</b> |



---

|                                           |           |
|-------------------------------------------|-----------|
| <b>B. Configuraciones de ESPnet-EZ</b>    | <b>64</b> |
| <b>C. Análisis exploratorio adicional</b> | <b>65</b> |

# Índice de figuras

|                                                          |    |
|----------------------------------------------------------|----|
| 5.1. Arquitectura general del sistema propuesto. . . . . | 38 |
| 5.2. Transformación afín específica de sesión. . . . .   | 40 |
| 5.3. Estructura del módulo adaptador profundo. . . . .   | 42 |

# Índice de tablas

|                                                                                                                                                     |    |
|-----------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 2.1. Progresión de los sistemas de decodificación de habla de vocabulario<br>abierto. WER: tasa de error por palabra. PPM: palabras por minuto. . . | 12 |
| 3.1. Comparativa de los conjuntos de datos evaluados: señal y tarea. . . . .                                                                        | 20 |
| 3.2. Comparativa de los conjuntos de datos evaluados: volumen y formato. .                                                                          | 20 |
| 3.3. Composición del conjunto Brain-to-Text '25 por corpus de origen. Datos<br>tomados de [lightbeam]. . . . .                                      | 24 |
| 5.1. Tratamiento de cada componente de OWSM v3.1 en el sistema propuesto.                                                                           | 38 |
| 5.2. Unidades de salida CTC implementadas. . . . .                                                                                                  | 43 |
| 5.3. Distribución de parámetros del sistema en la configuración de referencia.                                                                      | 46 |
| 6.1. Receta de entrenamiento de referencia. . . . .                                                                                                 | 50 |
| 6.2. Bloques experimentales y pregunta a la que responde cada uno. . . . .                                                                          | 55 |
| 7.1. Resultados por bloque experimental. . . . .                                                                                                    | 59 |

# Capítulo 1

## Introducción

### 1.1. Contexto

La comunicación verbal es una de las capacidades más básicas de la vida en sociedad y, sin embargo, puede perderse por completo mientras el resto de las funciones cognitivas permanecen intactas. Enfermedades neurodegenerativas como la esclerosis lateral amiotrófica (ELA), los ictus de tronco encefálico o determinadas lesiones medulares altas pueden provocar disartria severa, anartria o incluso el síndrome de cautiverio (*locked-in syndrome*): un estado en el que la persona conserva la conciencia y el lenguaje, pero carece de la vía motora necesaria para articularlo [card2024]. El intento de hablar sigue produciéndose en la corteza motora; lo que falla es el músculo, no el mensaje.

Las tecnologías de apoyo a la comunicación actualmente disponibles, como los teclados controlados por seguimiento ocular, permiten sortear parcialmente esa barrera, pero a un coste elevado en velocidad y esfuerzo: sus tasas de comunicación se sitúan en el orden de unas pocas palabras por minuto, frente a las aproximadamente 150 palabras por minuto del habla conversacional [willett2023]. Esa diferencia de más de un orden de magnitud no es un detalle técnico, sino la distancia que separa el poder deletrear una necesidad del poder mantener una conversación.

Las interfaces cerebro-computador (*Brain-Computer Interfaces*, BCI) abordan el problema desde el otro extremo del canal: en lugar de aprovechar el residuo motor que le quede al usuario, registran directamente la actividad cortical asociada a la intención y la traducen a una acción externa. Una BCI se compone, de forma general, de cuatro etapas: adquisición de la señal neuronal, extracción de características, decodificación mediante un modelo aprendido y realimentación al usuario. Las tecnologías de adquisición se ordenan en un compromiso claro entre invasividad y calidad de señal, que va desde la electroencefalografía (EEG) de superficie, no invasiva pero de baja relación

señal-ruido y escasa resolución espacial, hasta los registros intracorticales con matrices de microelectrodos implantadas en el tejido, que resuelven la actividad de poblaciones neuronales locales y son, hoy por hoy, la única modalidad con la que se han alcanzado prestaciones compatibles con una conversación [willett2023,card2024].

## 1.2. Del control motor al habla: los sistemas *brain-to-text*

Durante la última década, las BCI intracorticales han pasado de controlar cursores y brazos robóticos a decodificar lenguaje. Un primer salto cualitativo se produjo al decodificar la escritura manual imaginada, alcanzando unos 90 caracteres por minuto en un participante con tetraplejia [willett2021]. Poco después, dos trabajos simultáneos demostraron la decodificación del habla intentada: uno mediante electrocorticografía [metzger2023] y otro mediante matrices intracorticales en la corteza premotora ventral, con una tasa de error por palabra (*Word Error Rate*, WER) del 23,8 % sobre un vocabulario de 125.000 palabras a unas 62 palabras por minuto [willett2023].

El referente actual, y el que da origen a los datos empleados en este trabajo, es la neuroprótesis descrita por Card *et al.* [card2024]: en un participante con ELA avanzada y disartria severa, el sistema alcanzó un 99,6 % de acierto con un vocabulario de 50 palabras el primer día de uso y un 90,2 % sobre un vocabulario de 125.000 palabras el segundo día, sosteniendo posteriormente en torno al 97,5 % de acierto durante más de ocho meses y permitiendo conversaciones autónomas a unas 32 palabras por minuto a lo largo de más de 248 horas acumuladas de uso. Estos resultados sitúan el problema en una fase distinta: la viabilidad clínica ya no está en discusión, y la pregunta se desplaza hacia la eficiencia, la reproducibilidad y la generalización de los decodificadores.

Resulta especialmente relevante para este trabajo la forma que adopta la arquitectura de estos sistemas. Prácticamente todos ellos comparten la misma estructura: una secuencia de características neuronales muestreada a intervalos regulares se procesa mediante una red recurrente o un *transformer*, se entrena con una pérdida CTC (*Connectionist Temporal Classification*) [graves2006ctc] para emitir una secuencia de fonemas, y esa secuencia se convierte en texto mediante un modelo de lenguaje *n*-grama y, en los sistemas más recientes, un reordenamiento posterior con un modelo de lenguaje de gran tamaño. Es, punto por punto, la arquitectura clásica del reconocimiento automático del habla (*Automatic Speech Recognition*, ASR), con la única diferencia de que la entrada no es un espectrograma sino actividad cortical.

Esa coincidencia estructural plantea una pregunta natural que este trabajo toma como punto de partida. En procesamiento del lenguaje, visión por computador y re-

conocimiento del habla, el paradigma dominante consiste en preentrenar un modelo de gran capacidad sobre cantidades masivas de datos y adaptarlo después a la tarea concreta con una fracción mínima de datos y cómputo. En decodificación neuronal, en cambio, lo habitual sigue siendo entrenar el decodificador desde cero para cada participante, precisamente en el escenario donde los datos son más caros de obtener: cada hora de entrenamiento exige la colaboración activa de una persona enferma en una sesión clínica supervisada. Si un codificador de habla preentrenado hubiera aprendido algo genuinamente transferible sobre la estructura temporal de la producción del lenguaje, ese conocimiento debería poder reutilizarse aunque la modalidad de entrada cambie por completo.

### 1.3. Planteamiento del trabajo

Este Trabajo de Fin de Grado estudia la viabilidad de reutilizar un modelo de reconocimiento del habla preentrenado a gran escala como codificador de señal neuronal para una tarea *brain-to-text*. Concretamente, se adapta OWSM v3.1 (*Open Whisper-style Speech Model*) [peng2024owsmv31], una alternativa abierta y reproducible a Whisper [radford2023whisper] construida sobre la arquitectura E-Branchformer [kim2023branchformer], para decodificar a texto la actividad intracortical del conjunto de datos público del Brain-to-Text '25 [b2t25,card2024], registrada en el participante T15 mediante 256 microelectrodos a lo largo de 45 sesiones y 20 meses.

La adaptación requiere resolver un desajuste de modalidad: el modelo espera un banco de filtros mel derivado de audio y recibe, en su lugar, un vector de 512 características neuronales por ventana temporal. Para salvarlo se sustituye el *frontend* acústico original por un módulo adaptador entrenado desde cero, se antepone una transformación afín específica de cada sesión que absorbe la deriva de la señal entre días de registro, y se entrena el conjunto con un objetivo CTC puro, prescindiendo del decodificador autorregresivo del modelo original. Sobre esa base se estudian, mediante ablaciones controladas, las cuatro decisiones de diseño que determinan el comportamiento del sistema: cuánto aporta realmente el preentrenamiento, a qué granularidad lingüística conviene leer la corteza, cuántos parámetros hace falta adaptar y qué parte del conocimiento lingüístico acaba residiendo en el decodificador y qué parte en el modelo de lenguaje externo.

Conviene precisar desde el principio el carácter del trabajo. No se persigue competir con los sistemas que encabezan el *benchmark* oficial, que recurren a conjuntos de decenas de modelos y a modelos de lenguaje de gran tamaño para la fusión final de hipótesis, y cuyos requisitos de cómputo y memoria exceden con mucho los recursos disponibles aquí. El objetivo es distinto y complementario: producir evidencia expe-

rimental limpia y reproducible sobre *qué transfiere y qué no* de un modelo de habla a una modalidad que no es habla, obtenida íntegramente con herramientas de código abierto, datos públicos y una única GPU de gama media.

## 1.4. Motivación

### 1.4.1. Motivación clínica y social

La ELA presenta una incidencia aproximada de 2 casos por cada 100.000 habitantes y año en poblaciones europeas, y la mayoría de las personas afectadas desarrolla algún grado de deterioro del habla a lo largo de la enfermedad [longinetti2019]. A esta población se suman los supervivientes de ictus de tronco y otras lesiones neurológicas que cursan con anartria. Para todas ellas, la pérdida del habla no supone únicamente una limitación funcional: supone la pérdida de la autonomía en la relación con los demás,.

Las neuroprótesis del habla han demostrado ya que esa capacidad puede restituirse con calidad conversacional [card2024]. El cuello de botella se ha desplazado, por tanto, hacia cuestiones de ingeniería que condicionan su despliegue real: cuántos datos de calibración necesita el sistema, cuánto tarda en estar operativo tras el implante, y en qué medida sigue funcionando en una sesión nueva, un día distinto, sin recalibración. Cualquier avance que reduzca la cantidad de datos que hay que pedirle a un paciente enfermo tiene un impacto clínico directo, y el aprendizaje por transferencia es, precisamente, la familia de técnicas diseñada para ello.

### 1.4.2. Motivación científica y técnica

Desde el punto de vista técnico, el interés del trabajo reside en que plantea una pregunta que rara vez se formula de manera explícita: ¿qué es exactamente lo que aprende un modelo de habla preentrenado? La respuesta habitual, que aprende a reconocer voz, es demasiado gruesa. Un modelo como OWSM contiene, al menos, tres tipos de conocimiento superpuestos: un *frontend* acústico especializado en la señal de audio, un codificador que modela dependencias temporales de largo alcance en secuencias con estructura de lenguaje, y una cabeza de salida sobre un vocabulario de subpalabras que incorpora estadística ortográfica del inglés. Sustituir la entrada por actividad cortical desacopla estos tres componentes y permite medirlos por separado. Que las unidades de salida a nivel de carácter llegasen a superar a las subpalabras preentrenadas, por ejemplo, no sería un contratiempo experimental, sino una medida informativa sobre dónde reside y dónde no reside el conocimiento transferible.

Existe además un argumento neurocientífico que estructura buena parte de los ex-

perimentos. La corteza motora del habla codifica articulación, no ortografía. La unidad de salida a priori correcta es, por tanto, el fonema, y de hecho es la que utilizan tanto el sistema de referencia de este conjunto de datos como la práctica totalidad de las entradas del *benchmark*, que seleccionan modelo por tasa de error de fonema. Comparar sistemáticamente distintas granularidades de salida sobre un mismo eje de evaluación permite contrastar esa hipótesis con datos propios en lugar de asumirla.

### 1.4.3. Motivación de accesibilidad y reproducibilidad

Los sistemas que encabezan actualmente el *benchmark* combinan decenas de modelos entrenados con semillas distintas y emplean modelos de lenguaje de gran tamaño para la fusión de sus salidas, con unos requisitos de cómputo y de memoria que se detallan en el capítulo 2. Este trabajo se desarrolla íntegramente sobre datos públicos, bibliotecas de código abierto y una única GPU accesible mediante un servicio en la nube de bajo coste, y documenta de forma explícita el presupuesto de cómputo de cada experimento. Mostrar qué parte del problema sigue siendo abordable en esas condiciones tiene valor por sí mismo, tanto para grupos de investigación con recursos limitados como para el propio criterio de reproducibilidad de los resultados publicados.

### 1.4.4. Motivación personal

El interés personal por este trabajo surge de la confluencia de dos ámbitos que hasta ahora había abordado por separado a lo largo del grado: el procesamiento de señal y el aprendizaje profundo aplicado a secuencias, por un lado, y la aplicación de ambos a un problema con una finalidad médica tangible, por otro. Enfrentarse a un problema en el que el modelo no funciona hasta que se entienden simultáneamente la física de la señal, la arquitectura del modelo y la naturaleza lingüística de la salida ha resultado ser, además, la mejor forma posible de cerrar la formación del grado.

## 1.5. Objetivos

### 1.5.1. Objetivo general

El objetivo general de este trabajo es **evaluar la viabilidad de reutilizar un modelo de reconocimiento del habla preentrenado a gran escala (OWSM v3.1) como codificador de actividad neuronal intracortical en una tarea de decodificación *brain-to-text***, cuantificando de forma experimental qué componentes de dicho preentrenamiento transfieren a una modalidad no acústica, bajo qué configu-



ración de adaptación lo hacen, y cómo se reparte el conocimiento lingüístico entre el decodificador neuronal y el modelo de lenguaje externo.

### 1.5.2. Preguntas de investigación

El objetivo general se concreta en tres preguntas de investigación que vertebran el diseño experimental y la discusión de resultados:

**PI1. Transferencia.** ¿Qué parte de un modelo de habla preentrenado transfiere a una modalidad que no es habla? Es decir, ¿aporta el preentrenamiento de OWSM una ventaja medible frente a la misma arquitectura inicializada aleatoriamente, y cuántos parámetros del codificador es necesario adaptar para materializarla?

**PI2. Granularidad.** ¿A qué granularidad lingüística puede leerse la corteza motora del habla? ¿Cómo se comportan las unidades de salida de carácter, fonema, subpalabra y palabra cuando se comparan sobre un mismo eje de evaluación, y qué papel juega en ello el número de ejemplos de entrenamiento disponibles por clase?

**PI3. Localización del conocimiento lingüístico.** ¿Dónde reside el conocimiento lingüístico del sistema completo, en el codificador neuronal o en el modelo de lenguaje externo? ¿Es el modelo de lenguaje un mero reordenador de hipótesis acústicas o debe guiar activamente la búsqueda durante la decodificación?

### 1.5.3. Objetivos específicos

Para dar respuesta a lo anterior se plantean los siguientes objetivos específicos, cada uno asociado a un bloque experimental concreto:

**OE1. Construir una cadena de procesamiento y entrenamiento reproducible** que permita ingerir los registros neuronales del Brain-to-Text '25 en formato HDF5 y entrenar sobre ellos un modelo OWSM mediante ESPnet-EZ [someki2024espnetez,watanabe2018espnet], incluyendo la verificación de integridad de los datos, la parametrización completa de cada experimento mediante identificadores trazables y el registro sistemático de resultados y curvas de entrenamiento.

**OE2. Diseñar el módulo de adaptación de modalidad** que sustituya al *frontend* acústico del modelo, de forma que las 512 características neuronales por ventana temporal se proyecten al espacio de entrada esperado por el codificador, e incorporar una capa específica de sesión, en forma de transformación afín aprendida por cada día de registro, que compense la deriva de la señal entre sesiones.

- OE3. Cuantificar la aportación real del preentrenamiento** (PI1) mediante un control científico explícito que compare, con presupuesto de cómputo idéntico, tres condiciones: codificador preentrenado y ajustado, codificador preentrenado y congelado, y codificador con inicialización aleatoria y ajustado.
- OE4. Determinar la granularidad de salida más adecuada** (PI2) comparando objetivos CTC de carácter, fonema, subpalabra específica del corpus, palabra de vocabulario cerrado y subpalabra original del modelo preentrenado, evaluando todas las variantes sobre la tasa de error por palabra del texto finalmente decodificado, que constituye el único eje de comparación común entre unidades distintas.
- OE5. Caracterizar el compromiso entre capacidad y coste del módulo adaptador**, comparando arquitecturas de complejidad creciente bajo una receta de entrenamiento idéntica y reportando, junto al rendimiento, el número de parámetros y el tiempo de entrenamiento por época, de modo que las comparaciones queden libres de las confusiones experimentales presentes en las primeras iteraciones del trabajo.
- OE6. Analizar la relación entre parámetros entrenables y rendimiento** (PI1) mediante estrategias de adaptación de coste creciente, desde la adaptación de bajo rango (LoRA) [hu2022lora] sobre un codificador congelado hasta el descongelado progresivo de sus capas superiores, obteniendo una curva que permita identificar el punto de operación más eficiente.
- OE7. Evaluar la generalización a sesiones no observadas**, escenario que reproduce la situación real de despliegue clínico, midiendo la degradación del sistema sobre sesiones excluidas del entrenamiento y cuantificando el compromiso que introduce la capa específica de sesión, así como el coste de una adaptación mínima a una sesión nueva.
- OE8. Determinar el papel del modelo de lenguaje externo** (PI3) integrando un decodificador CTC por búsqueda en haz con un modelo de lenguaje  $n$ -grama [heafield2011kenlm], explorando el espacio de sus hiperparámetros de fusión, comparando distintos órdenes del modelo y contrastando su rendimiento con el límite superior que impone la lista de hipótesis acústicas (análisis *oracle*), lo que permite discriminar si el modelo de lenguaje reordena o guía la búsqueda.
- OE9. Consolidar la configuración final y estimar su variabilidad**, entrenando el mejor sistema resultante con varias semillas de inicialización para reportar los resultados con una medida de dispersión que permita distinguir las diferencias reales entre configuraciones del ruido experimental, y situar dichos resultados

en el contexto del sistema de referencia oficial del conjunto de datos.

#### 1.5.4. Alcance y limitaciones

Para delimitar con precisión qué se puede y qué no se puede concluir de este trabajo, se hacen explícitas desde el inicio las siguientes restricciones:

- **Un único participante.** Todos los experimentos se realizan sobre los datos del participante T15. Las conclusiones sobre transferencia y granularidad son, por tanto, específicas de este sujeto, de la localización de sus implantes y del inglés como lengua objetivo, y no pueden extrapolarse sin más a otros participantes o idiomas.
- **Decodificación fuera de línea.** El sistema se evalúa en diferido sobre grabaciones ya adquiridas. No se aborda la ejecución en tiempo real ni la latencia percibida por el usuario, ambas condiciones necesarias para un uso conversacional.
- **Presupuesto de cómputo acotado.** El trabajo se ejecuta sobre una única GPU de gama media en un entorno en la nube. Esto excluye por construcción las técnicas de conjunto de decenas de modelos y el uso de modelos de lenguaje de gran tamaño para la fusión de hipótesis, que son las que sostienen los primeros puestos del *benchmark*. Los resultados absolutos deben leerse teniendo esto presente.
- **Modelo de lenguaje propio.** El modelo de lenguaje  $n$ -grama oficial de la competición requiere una cantidad de memoria principal que excede la disponible en el entorno de trabajo, por lo que se entrena un modelo propio de orden reducido sobre las transcripciones del corpus. Esta restricción condiciona los valores absolutos de tasa de error por palabra, aunque no invalida las comparaciones relativas, que se realizan siempre con el mismo modelo de lenguaje.
- **Ablaciones con presupuesto igualado, no hasta convergencia.** Las comparaciones entre configuraciones se realizan asignando a todas las variantes el mismo número de épocas y desactivando la parada temprana, de modo que la comparación sea justa. Los valores reportados en las ablaciones no corresponden, por tanto, al rendimiento máximo alcanzable por cada configuración, sino a su rendimiento bajo un presupuesto común.

## 1.6. Estructura de la memoria

El resto de la memoria se organiza como sigue. El capítulo 2 revisa el estado del arte en neuroprótesis del habla y en la adaptación de modelos de habla preentrenados, y sitúa la aportación de este trabajo respecto a ambas líneas. El capítulo 3 justifica la elección del conjunto de datos frente a las alternativas evaluadas, lo describe en detalle y analiza los retos que impone. El capítulo 4 desarrolla los fundamentos de los modelos y algoritmos empleados: el codificador atencional, el objetivo CTC, la adaptación de rango bajo, la decodificación con modelo de lenguaje y las métricas. El capítulo 5 describe el sistema propuesto, detallando qué se conserva del modelo preentrenado, qué se sustituye y qué se añade. El capítulo 6 expone el protocolo experimental, de evaluación y de comparación, junto con el diseño de los bloques de experimentos. El capítulo 7 recoge y discute los resultados obtenidos. El capítulo 8 reúne las limitaciones del estudio y las líneas de continuación que se consideran más prometedoras. Finalmente, el capítulo 9 resume las conclusiones del trabajo.

# Capítulo 2

## Estado del arte

Este capítulo revisa el contexto científico y técnico en el que se inscribe el trabajo. Se organiza en tres bloques. El primero (apartados 2.1 a 2.4) recorre la evolución de las neuroprótesis del habla hasta los sistemas actuales y disecciona la arquitectura que todos ellos comparten. El segundo (apartados 2.5 y 2.6) resume el estado del arte en modelos de habla preentrenados y en las técnicas de adaptación eficiente que hacen viable su reutilización. El tercero (apartados 2.7 y 2.8) revisa los trabajos que han intentado explícitamente cruzar ambos campos y sitúa la aportación de esta memoria respecto a ellos.

### 2.1. Modalidades de registro en interfaces cerebro-computador

La elección de la técnica de registro condiciona por completo el tipo de información que puede extraerse y, en consecuencia, la clase de decodificador que tiene sentido construir. Las modalidades empleadas en BCI del habla se ordenan en un compromiso entre invasividad y calidad de señal:

**Electroencefalografía (EEG).** No invasiva, de bajo coste y ampliamente disponible. Registra la actividad sumada de grandes poblaciones neuronales a través del cráneo, lo que impone una resolución espacial de centímetros y una relación señal-ruido baja. Es adecuada para paradigmas de selección discreta, pero la decodificación de habla continua a partir de EEG sigue siendo objeto de debate metodológico, en particular en lo relativo a la independencia entre particiones y a la elección de métricas que premian la similitud semántica frente a la recuperación literal del estímulo.

**Estereoelectroencefalografía (sEEG).** Electrodos de profundidad implantados con

finés diagnósticos, habitualmente en pacientes con epilepsia refractaria. Ofrecen buena resolución temporal y acceso a estructuras profundas, pero su colocación responde a criterios clínicos y no a la conveniencia del decodificador, y los registros suelen ser de corta duración.

**Electrocorticografía (ECoG).** Rejillas de electrodos colocadas sobre la superficie cortical, sin penetrar en el tejido. Combinan una cobertura espacial amplia con una estabilidad crónica notable, y sobre ellas se construyeron los primeros sistemas de habla en tiempo real [moses2021,metzger2023].

**Matrices intracorticales de microelectrodos.** Conjuntos de electrodos penetrantes (típicamente matrices tipo Utah) que registran la actividad de poblaciones neuronales locales con resolución de unidad múltiple. Es la modalidad de mayor riesgo quirúrgico y, a la vez, la única con la que se han alcanzado prestaciones compatibles con la conversación [willett2023,card2024]. Es también la modalidad de los datos empleados en este trabajo.

De las características extraídas de estos registros, dos se han impuesto como estándar en las matrices intracorticales: el recuento de cruces de umbral (*threshold crossings*), que cuenta los eventos en que la señal filtrada cae por debajo de un umbral proporcional al valor eficaz del ruido, y la potencia en la banda de espigas (*spike band power*). Ambas se agregan en ventanas de 20 ms, lo que produce una secuencia a 50 Hz [b2t25].

## 2.2. Evolución de las neuroprótesis del habla

Los primeros sistemas de comunicación basados en BCI operaban por selección: el usuario elegía letras o comandos de un conjunto discreto. El salto hacia la producción continua de lenguaje llegó por dos vías simultáneas.

La primera fue la decodificación de la escritura manual imaginada, que alcanzó unos 90 caracteres por minuto con una precisión superior al 99% empleando corrección automática [willett2021]. Aunque el gesto decodificado no es el habla, ese trabajo introdujo dos elementos que siguen siendo estándar: el entrenamiento con pérdida CTC sobre secuencias sin alineamiento explícito, y las capas de entrada específicas de cada día de registro para compensar la deriva de la señal.

La segunda vía atacó directamente el habla intentada. Moses *et al.* [moses2021] demostraron la decodificación en tiempo real en una persona con anartria mediante ECoG, con un vocabulario cerrado de 50 palabras. Dos años después, dos trabajos publicados simultáneamente ampliaron el problema a vocabulario abierto: Metzger *et al.* [metzger2023] con ECoG de alta densidad, incorporando síntesis de voz y control de un

avatar, y Willett *et al.* [willett2023] con matrices intracorticales en la corteza premo-tora ventral, alcanzando un 23,8 % de tasa de error por palabra sobre un vocabulario de 125.000 palabras a unas 62 palabras por minuto.

El referente actual es el sistema descrito por Card *et al.* [card2024], del que proceden los datos de este trabajo. Sus resultados marcan un cambio de fase respecto a los anteriores: 99,6 % de acierto con vocabulario de 50 palabras el primer día de uso, 90,2 % sobre 125.000 palabras el segundo día tras 1,4 horas adicionales de datos, y en torno al 97,5 % de acierto sostenido durante más de ocho meses, con más de 248 horas acumuladas de conversación autónoma a unas 32 palabras por minuto. La cuadro 2.1 resume esta progresión.

**Tabla 2.1:** Progresión de los sistemas de decodificación de habla de vocabulario abierto. WER: tasa de error por palabra. PPM: palabras por minuto.

| Trabajo                             | Registro      | Participante    | Vocabulario    | WER    | PPM    |
|-------------------------------------|---------------|-----------------|----------------|--------|--------|
| Moses <i>et al.</i> [moses2021]     | ECoG          | Anartria        | 50 palabras    | —      | ~15    |
| Willett <i>et al.</i> [willett2021] | Intracortical | Tetraplejia     | Escritura      | < 1 %* | 90 cpm |
| Metzger <i>et al.</i> [metzger2023] | ECoG          | Ictus de tronco | 1.024 palabras | —      | ~78    |
| Willett <i>et al.</i> [willett2023] | Intracortical | T12 (ELA)       | 125.000        | 23,8 % | 62     |
| Card <i>et al.</i> [card2024]       | Intracortical | T15 (ELA)       | 125.000        | ~2,5 % | 32     |

\* Tasa de error por carácter con corrección automática, no WER. CPM: caracteres por minuto.

## 2.3. Anatomía de un decodificador *brain-to-text*

Pese a la diversidad de grupos y modalidades, los sistemas anteriores comparten una arquitectura común que conviene desglosar, porque es exactamente la arquitectura de un sistema de reconocimiento automático del habla y es esa coincidencia la que motiva este trabajo.

A lo largo de esta sección se hace referencia repetidamente a los dos *benchmarks* públicos del campo, Brain-to-Text '24 y Brain-to-Text '25, y a sus respectivos sistemas de referencia, por ser el punto de comparación común de la literatura reciente. Ambos se describen en detalle en la apartado 2.4; basta por ahora con saber que son conjuntos de datos abiertos con particiones fijas y clasificación pública, y que el sistema de referencia de los dos es la red recurrente descrita a continuación.

### 2.3.1. Características de entrada y no estacionariedad

La entrada es una secuencia de vectores de características neuronales a 50 Hz. Un problema transversal, y probablemente el más limitante en la práctica, es la **no estacionariedad**: la relación entre actividad registrada e intención cambia entre sesiones

por micromovimientos del implante, gliosis, variaciones en el estado del participante y degradación progresiva del electrodo. Un decodificador entrenado con datos de una sesión se degrada de forma apreciable en la siguiente. La solución estándar, introducida en [willett2021] y presente en prácticamente todos los sistemas posteriores, consiste en aprender una transformación de entrada específica de cada sesión (típicamente afín) que proyecta los datos de cada día a un espacio común, dejando que el resto del modelo sea compartido. Los sistemas más recientes refinan esta idea con proyecciones jerárquicas de bajo rango a nivel de mes y de día [brainwhisperer].

### 2.3.2. El codificador neuronal

El codificador transforma la secuencia de características en una secuencia de distribuciones sobre unidades lingüísticas. El modelo de referencia de ambos *benchmarks* es una red recurrente con unidades GRU, y resulta llamativo lo difícil que ha sido superarla: los intentos de sustituirla por modelos de espacio de estados de tipo Mamba igualaron la pérdida CTC y la tasa de error de fonema, pero quedaron por detrás en tasa de error por palabra, y las variantes basadas en Transformer tampoco mejoraron de forma clara el modelo recurrente [b2t24lessons]. Se ha sugerido que la tarea no depende de forma significativa de la información de largo alcance, lo que retiraría la principal ventaja de las arquitecturas atencionales. Trabajos posteriores han obtenido resultados competitivos con codificadores Conformer [khanday2026], de modo que la cuestión no está cerrada.

### 2.3.3. Objetivo de entrenamiento y unidad lingüística

El entrenamiento se realiza casi universalmente con la pérdida CTC [graves2006ctc], que permite aprender a partir de pares secuencia-etiqueta sin alineamiento temporal explícito, introduciendo un símbolo en blanco y marginalizando sobre todos los alineamientos compatibles con la etiqueta. Es la elección natural aquí, porque no se dispone de la correspondencia temporal entre actividad cortical y unidades emitidas: en un participante con parálisis no hay audio de referencia con el que alinear.

La unidad de salida dominante es el **fonema**, con la justificación neurocientífica de que la corteza motora del habla codifica articulación y no ortografía. Es la unidad del sistema de referencia de ambos *benchmarks* y la que emplean casi todas las entradas, que además seleccionan modelo por tasa de error de fonema. Se han explorado alternativas de granularidad más fina: el equipo ganador del Brain-to-Text '24 obtuvo su mejora principal empleando difonos, es decir, transiciones entre fonemas, con el argumento de que capturan mejor la coarticulación [b2t24lessons]. En el extremo opuesto,



algunos trabajos recientes han demostrado que la decodificación directa a nivel de **carácter** es viable de forma totalmente extremo a extremo [khanday2026], lo que resulta especialmente relevante para esta memoria.

### 2.3.4. El modelo de lenguaje y la formación de palabras

La salida del codificador es una secuencia de distribuciones sobre unidades sublexicales que, por sí sola, produce texto con una tasa de error inaceptable. La conversión a texto se realiza mediante una búsqueda que combina esa evidencia acústica con un modelo de lenguaje. El procedimiento estándar en los sistemas de referencia es un decodificador ponderado de estados finitos que integra un modelo  $n$ -grama de orden alto para generar una lista de hipótesis, seguido de una segunda etapa de reordenamiento con un modelo de lenguaje de gran tamaño [b2t24lessons].

Esta etapa aporta una parte muy sustancial del rendimiento final del sistema, hasta el punto de que la comparación entre codificadores sólo es significativa si se realiza con el mismo decodificador lingüístico. Tiene además un coste de recursos considerable: el modelo de 5-gramas empleado en el *benchmark* requiere del orden de 300 GB de memoria principal y el de 3-gramas unos 60 GB, cifras fuera del alcance de un entorno de trabajo estándar, lo que ha llevado a varios participantes a entrenar modelos propios de orden reducido [cheng2026]. Existen implementaciones más eficientes en memoria específicamente diseñadas para este escenario [lightbeam].

La literatura emplea el modelo de lenguaje en dos regímenes cualitativamente distintos, el reordenamiento de una lista de hipótesis ya generada y la fusión superficial durante la propia búsqueda. La distinción es central en esta memoria y se formaliza en la apartado 4.6.4.

## 2.4. Los *benchmarks* Brain-to-Text

La existencia de conjuntos de datos públicos con particiones fijas y métrica común ha sido determinante para la aceleración reciente del campo.

El **Brain-to-Text '24** se construyó sobre datos del participante T12: 12.100 frases registradas en 25 sesiones a lo largo de cuatro meses con matrices de 128 electrodos en la corteza motora ventral [lightbeam]. Su modelo de referencia, la red recurrente con decodificación en dos etapas descrita anteriormente, alcanzó un 9,7% de tasa de error por palabra, mientras que la mejor entrada de la competición llegó al 5,8% [b2t24lessons].

El **Brain-to-Text '25**, empleado en este trabajo, amplía considerablemente la es-

cala: 10.948 frases del participante T15 registradas en 45 sesiones a lo largo de 20 meses con 256 microelectrodos, agrupados en cuatro matrices de alta densidad de 64 canales cada una, de las que se extraen 512 características por ventana de 20 ms [b2t25,card2024]. El horizonte temporal de 20 meses convierte la no estacionariedad en un problema de primer orden, muy por encima de lo que planteaba el conjunto anterior.

Las entradas que encabezan la clasificación de este segundo *benchmark* recurren de forma sistemática a conjuntos de modelos y a modelos de lenguaje de gran tamaño para la fusión de sus salidas: una de ellas combina 29 codificadores preentrenados inicializados con semillas distintas y emplea GPT-4 ajustado para fusionar las hipótesis de fonemas y frases de todos ellos [crossspecies2025]. Este dato es relevante para interpretar cualquier resultado individual: la distancia entre un sistema de modelo único y la cabeza de la clasificación no es únicamente una diferencia de arquitectura, sino también de presupuesto de cómputo.

## 2.5. Modelos de habla preentrenados

En paralelo, el reconocimiento automático del habla ha experimentado su propia transformación. El paradigma actual consiste en preentrenar un modelo de gran capacidad sobre volúmenes masivos de audio y adaptarlo después a la tarea concreta.

Dentro de este paradigma conviven dos familias. La primera es la del **aprendizaje autosupervisado**, representada por wav2vec 2.0, HuBERT y WavLM, que aprenden representaciones a partir de audio no etiquetado mediante objetivos contrastivos o de predicción enmascarada, y requieren una etapa posterior de ajuste supervisado. La segunda es la de la **supervisión débil a gran escala**, cuyo exponente es Whisper [radford2023whisper], entrenado sobre 680.000 horas de audio con transcripción de calidad heterogénea recogidas de la web, y que funciona directamente como sistema de reconocimiento multilingüe sin ajuste adicional.

El principal inconveniente de Whisper para la investigación es que ni sus datos de entrenamiento ni su procedimiento son plenamente reproducibles. **OWSM** (*Open Whisper-style Speech Model*) [peng2023owsm] nació precisamente para cubrir ese hueco: reproduce el estilo de entrenamiento de Whisper empleando exclusivamente conjuntos de datos públicos y una herramienta de código abierto, ESPnet [watanabe2018espnet], publicando datos, recetas y pesos. La versión v3.1 [peng2024owsmv31] sustituye el codificador Transformer por la arquitectura E-Branchformer [kim2023ebranchformer], que combina una rama de autoatención con una rama convolucional para capturar simultáneamente dependencias globales y patrones locales, y mejora tanto la precisión como la velocidad de inferencia respecto a la versión anterior. Arquitectónicamente, OWSM es

un modelo codificador-decodificador entrenado con un objetivo conjunto CTC-atención [kim2017jointctc], lo que resulta conveniente aquí: la rama CTC puede utilizarse de forma aislada, prescindiendo por completo del decodificador autorregresivo.

## 2.6. Adaptación eficiente en parámetros

Ajustar la totalidad de los parámetros de un modelo preentrenado sobre un conjunto de datos pequeño es caro y propenso al sobreajuste. Las técnicas de adaptación eficiente en parámetros (*Parameter-Efficient Fine-Tuning*, PEFT) abordan el problema modificando únicamente una fracción reducida de los pesos.

La más extendida es **LoRA** [hu2022lora], que congela los pesos originales e introduce, en paralelo a las matrices de proyección seleccionadas, un par de matrices de rango reducido cuyo producto se suma a la salida. La hipótesis subyacente es que la actualización necesaria para adaptarse a una tarea nueva tiene rango intrínseco bajo. Alternativas habituales son los módulos adaptadores insertados entre capas, el ajuste de prefijos y, en su versión más simple, el **descongelado parcial**: mantener congeladas las capas inferiores y ajustar únicamente las superiores.

Estas técnicas comparten una condición de aplicabilidad que condiciona el diseño experimental de esta memoria: presuponen un modelo base congelado, ya que su beneficio no procede de añadir capacidad sino de restringir la adaptación. El argumento formal se desarrolla en la apartado 4.5.

## 2.7. Transferencia de modelos de habla a señal neuronal

La idea de reutilizar un modelo entrenado sobre audio para decodificar actividad cerebral no es nueva, y conviene revisar con detalle los trabajos previos porque delimitan con precisión qué queda por responder.

Una primera línea emplea modelos de habla como **espacio de representación objetivo**. Kim *et al.* [braintalker] entrenan un codificador que proyecta señal ECoG al espacio latente de wav2vec 2.0, para sintetizar después voz inteligible en un escenario de muy pocos datos. En una variante próxima, se ha reentrenado la arquitectura de wav2vec sobre registros ECoG no etiquetados y se han empleado las representaciones resultantes como entrada de un decodificador supervisado, con mejoras consistentes respecto a usar la señal original y con transferencia positiva incluso entre pacientes distintos [ecogssl2024]. En estos trabajos, sin embargo, lo que se transfiere es la *arqui-*

*tectura* o el *objetivo autosupervisado*, no los pesos aprendidos sobre audio.

La línea directamente relevante para esta memoria es la que **transfiere los pesos preentrenados sobre habla** sustituyendo la entrada acústica por señal neuronal. Fiedler *et al.* [fiedler2025wav2vec2brain] reemplazan el extractor de características de audio de wav2vec 2.0 por un extractor de características neuronales entrenado desde cero y comparan sistemáticamente tres condiciones sobre los datos del participante T12: ajuste completo partiendo de los pesos preentrenados, entrenamiento desde cero con la misma arquitectura, y modelo preentrenado congelado con únicamente el extractor entrenable. El ajuste completo con pesos preentrenados obtiene la mejor tasa de error por carácter, 18,54 %, superando en 20,46 puntos porcentuales al entrenamiento desde cero y en 15,92 puntos a la variante congelada. Es, hasta donde alcanza esta revisión, la evidencia más directa de que existe transferencia real desde el reconocimiento del habla hacia la decodificación neuronal.

Más recientemente, BrainWhisperer [brainwhisperer] lleva esta idea a Whisper y al escenario intracortical de 256 canales. Su punto de partida es un hallazgo de interpretabilidad: el codificador de Whisper aprende representaciones selectivas de fonema con atención localizada, lo que sugiere que sus capas iniciales son un extractor articulatorio reutilizable. Sobre esa base construyen un modelo con pérdida híbrida, CTC de fonemas tomada de la tercera capa del codificador y entropía cruzada sobre unidades de palabra, atención propia con ventana para capturar continuidad articulatoria, proyecciones jerárquicas de bajo rango por mes y por día frente a la no estacionariedad, e incrustaciones específicas de sujeto que habilitan el entrenamiento conjunto entre participantes. El sistema se sitúa entre las mejores entradas del Brain-to-Text '25.

Existe además una tercera línea, complementaria, que preentrena **modelos fundacionales específicos de señal neuronal** en lugar de reutilizar modelos de habla: encoders entrenados sobre múltiples tareas motoras, participantes e incluso especies, que se ajustan después a la tarea de habla [crossspecies2025]. Esta aproximación evita el desajuste de modalidad, a costa de requerir la agregación de grandes volúmenes de datos neuronales.

## 2.8. Posicionamiento de este trabajo

De la revisión anterior se desprenden tres observaciones que delimitan la aportación de esta memoria.

En primer lugar, **la transferencia desde modelos de habla a señal neuronal está demostrada, pero poco caracterizada**. El trabajo de Fiedler *et al.* [fiedler2025wav2vec2brain] establece que existe, sobre wav2vec 2.0 y el participante T12.

BrainWhisperer [brainwhisperer] la explota con éxito sobre Whisper, pero como parte de un sistema con múltiples modificaciones simultáneas orientado a maximizar el rendimiento en la competición, lo que hace difícil atribuir la mejora a un componente concreto. Falta, entre ambos, un estudio de ablación con presupuesto de cómputo igualado que aisle cada decisión de diseño por separado sobre el conjunto de datos más reciente.

En segundo lugar, **la elección de la unidad lingüística de salida rara vez se justifica empíricamente**. El fonema se adopta por argumento neurocientífico y por continuidad con el sistema de referencia, mientras que el carácter aparece en trabajos aislados [khanday2026]. No se ha encontrado una comparación sistemática de granularidades sobre un mismo modelo, un mismo presupuesto y un mismo eje de evaluación, que es la única forma de contrastar ese argumento en lugar de asumirlo.

En tercer lugar, **los resultados publicados son difíciles de reproducir con recursos modestos**. Los sistemas de cabecera combinan decenas de modelos y modelos de lenguaje de gran tamaño, y hasta el modelo  $n$ -grama de referencia excede la memoria disponible en un entorno estándar [cheng2026]. Documentar qué parte del rendimiento sigue siendo accesible con una única GPU de gama media y un modelo de lenguaje entrenado sobre el propio corpus tiene, por tanto, valor metodológico.

Este trabajo se sitúa en la intersección de las tres observaciones. Adapta OWSM v3.1 [peng2024owsmv31] —y no Whisper, por ser un modelo íntegramente reproducible, con datos y recetas públicos— a los datos del Brain-to-Text '25, y en lugar de perseguir la mejor cifra posible, ejecuta un conjunto de ablaciones controladas, con una única variable modificada por experimento y presupuesto de cómputo idéntico, orientadas a responder las tres preguntas de investigación formuladas en el capítulo 1: qué transfiere del preentrenamiento, a qué granularidad conviene leer la corteza, y dónde reside el conocimiento lingüístico del sistema.

Como punto de referencia directo para los resultados obtenidos, resulta especialmente útil el trabajo de Khanday *et al.* [khanday2026], que emplea el mismo conjunto de datos, la misma unidad de salida a nivel de carácter, una capa de alineamiento específica de sesión análoga a la utilizada aquí y decodificación CTC voraz sin modelo de lenguaje externo, alcanzando una tasa de error por carácter del 23,80 % sobre la partición de validación. Al compartir métrica, datos y condiciones de decodificación, constituye la comparación más informativa disponible para un codificador entrenado desde cero, y por tanto una referencia natural frente a la que contrastar la hipótesis de transferencia que motiva esta memoria.

# Capítulo 3

## Conjunto de datos

La elección del conjunto de datos condiciona el resto del trabajo: determina qué preguntas pueden formularse, qué arquitecturas son viables y con qué resultados publicados es posible compararse. Este capítulo justifica esa elección frente a las alternativas evaluadas, describe en detalle el conjunto finalmente seleccionado y analiza los retos específicos que plantea, varios de los cuales explican decisiones de diseño posteriores.

### 3.1. Criterios de selección

Antes de fijar el conjunto de trabajo se evaluaron cuatro corpus públicos de decodificación de habla a partir de señal cerebral. Los criterios aplicados fueron los siguientes:

- C1. Tarea compatible con el modelo.** El sistema propuesto reutiliza un codificador de reconocimiento del habla entrenado sobre secuencias largas. Requiere, por tanto, un corpus de **frases continuas**, no de palabras o sílabas aisladas: un modelo diseñado para modelar dependencias temporales de varios segundos no puede evaluarse sobre estímulos de dos segundos.
- C2. Salida textual.** El objetivo es *brain-to-text*, no *brain-to-speech*. Los corpus orientados a la reconstrucción de audio requieren una cadena de síntesis (mel-espectrograma más vocoder) y un tipo de evaluación distinto, ajenos a este trabajo.
- C3. Volumen suficiente.** El ajuste de un modelo de más de cien millones de parámetros exige un número de muestras que descarta los corpus de pocos minutos por sujeto.
- C4. Relación señal-ruido.** La hipótesis de partida —que un codificador de habla puede extraer estructura temporal de la señal neuronal— sólo es contrastable si

la señal contiene esa estructura. Las modalidades de baja resolución espacial la comprometen.

- C5. Comparabilidad.** La existencia de particiones fijas, métrica oficial y resultados publicados permite situar los resultados propios en un contexto objetivo, en lugar de reportarlos de forma aislada.
- C6. Coste de acceso y preparación.** Formato documentado, código de carga disponible y datos ya organizados en particiones reducen el tiempo dedicado a ingeniería de datos, escaso en un trabajo de esta duración.

## 3.2. Alternativas evaluadas

La cuadro 3.1 resume los cuatro conjuntos considerados.

**Tabla 3.1:** Comparativa de los conjuntos de datos evaluados: señal y tarea.

| Conjunto          | Tipo de señal                         | Sujetos | Nivel de tarea        |
|-------------------|---------------------------------------|---------|-----------------------|
| UGR-MINDVOICE     | EEG (64 canales)                      | 15      | Palabras y sílabas    |
| sEEG2Speech       | sEEG                                  | 3       | Frases (5–7 palabras) |
| SingleWord (NL)   | sEEG (1.103 canales)                  | 10      | Palabras aisladas     |
| Brain-to-Text '25 | Microelectrodos intracorticales (256) | 1       | Frases continuas      |

**Tabla 3.2:** Comparativa de los conjuntos de datos evaluados: volumen y formato.

| Conjunto          | Volumen                                    | Salida objetivo       | Formato              |
|-------------------|--------------------------------------------|-----------------------|----------------------|
| UGR-MINDVOICE     | 60 palabras, 30 pseudopalabras, 95 sílabas | Clasificación o texto | MNE/EDF              |
| sEEG2Speech       | 100 frases, 10–20 min por sujeto           | Audio                 | Propio (OSF)         |
| SingleWord (NL)   | 100 palabras, ~5 min por sujeto            | Espectrograma o audio | BIDS / NWB           |
| Brain-to-Text '25 | 10.948 frases, 45 sesiones en 20 meses     | Texto                 | HDF5 con particiones |

### 3.2.1. UGR-MINDVOICE

Corpus multimodal de EEG de superficie con 15 participantes nativos de español, que incluye tanto habla pronunciada (*overt*) como imaginada (*covert*), con estímulos de

95 sílabas consonante-vocal que cubren el inventario fonético del español, 60 palabras de seis categorías semánticas y 30 pseudopalabras. Es el único corpus en español para esta tarea y su diseño experimental es cuidadoso, con datos y código públicos.

Fue descartado por dos razones. La primera es el nivel de la tarea: los estímulos son palabras y sílabas aisladas, incompatibles con el criterio C1. La segunda es la modalidad: el EEG de superficie tiene una resolución espacial y una relación señal-ruido muy inferiores a las de los registros invasivos, lo que compromete el criterio C4 y, con él, la **interpretabilidad de un resultado negativo**. Si el sistema fracasase, no sería posible distinguir entre ausencia de transferencia desde el modelo de habla y ausencia de señal decodificable en la entrada, que son conclusiones muy distintas. El descarte responde, por tanto, al diseño experimental de este trabajo concreto y no constituye un juicio sobre la calidad del corpus, cuyo diseño es cuidadoso y cuya orientación a un idioma distinto del inglés cubre un hueco real.

### 3.2.2. sEEG2Speech

Corpus de estereoelectroencefalografía en tres pacientes con epilepsia, que leen 100 frases de cinco a siete palabras en neerlandés. La señal se filtra en la banda gamma alta (70–170 Hz) y el objetivo es la reconstrucción directa de audio mediante mel-espectrogramas y un vocoder.

Cumple el criterio C1 pero incumple el C2 —el objetivo es audio, no texto— y sobre todo el C3: entre diez y veinte minutos por sujeto, de los cuales una fracción considerable es silencio, es un volumen incompatible con el ajuste de un modelo de gran tamaño.

### 3.2.3. SingleWordProductionDutch

Diez pacientes con epilepsia y 1.103 canales sEEG en total, leyendo 100 palabras aisladas en neerlandés, con formato BIDS bien documentado y una cadena de referencia basada en filtrado en gamma alta, reducción dimensional y regresión hacia el log-mel espectrograma.

Es el corpus con más participantes de los evaluados, lo que lo haría interesante para estudios de generalización entre sujetos, pero incumple C1 (palabras aisladas), C2 (salida de audio) y C3 (unos cinco minutos por sujeto).



### 3.2.4. Brain-to-Text '25

Es el único de los cuatro que satisface todos los criterios simultáneamente: frases continuas, salida textual, volumen de casi once mil frases, registros intracorticales de alta relación señal-ruido, particiones oficiales con clasificación pública y resultados publicados, y datos organizados en HDF5 con código de carga proporcionado por los autores. Es, en consecuencia, el conjunto empleado en este trabajo.

Conviene señalar que esta elección tiene un coste evidente: **un único participante**. Las conclusiones obtenidas son específicas de T15, de la localización de sus implantes y del inglés, y no pueden extrapolarse sin verificación a otros sujetos. Se asume este compromiso porque los tres corpus alternativos, aunque más diversos en participantes, no permiten formular la pregunta de investigación de este trabajo.

## 3.3. El conjunto Brain-to-Text '25

### 3.3.1. Origen y participante

Los datos proceden del ensayo clínico descrito por Card *et al.* [card2024] y se publicaron posteriormente como *benchmark* abierto y competición [b2t25]. El participante, identificado como T15, es un varón con esclerosis lateral amiotrófica avanzada y disartria severa, con el habla ininteligible pero con la corteza motora del habla funcionalmente intacta.

El protocolo es una tarea de copia: se presenta una frase en pantalla y, tras una señal de inicio, el participante intenta pronunciarla a su propio ritmo. Se registra la actividad neuronal durante ese intento. Es importante subrayar que **no existe audio de referencia** con el que alinear la señal, ya que la parálisis impide la producción de habla inteligible; de ahí que el entrenamiento deba recurrir necesariamente a un objetivo que no requiera alineamiento explícito, como CTC.

### 3.3.2. Adquisición y características neuronales

El registro se realiza mediante cuatro matrices de microelectrodos de alta densidad de 64 canales cada una, implantadas en la corteza motora del habla del hemisferio izquierdo, lo que suma 256 electrodos [b2t25].

De cada electrodo se extraen dos características, lo que produce un vector de **512 dimensiones por ventana temporal**:

**Cruces de umbral** (*threshold crossings*). Recuento de eventos en que la señal fil-

trada cruza un umbral fijado en  $-4,5$  veces el valor eficaz del ruido del canal. Es una aproximación a la tasa de disparo de la población neuronal próxima al electrodo, sin necesidad de clasificar espigas individuales.

**Potencia en la banda de espigas (*spike band power*).** Energía de la señal en la banda de frecuencia asociada a la actividad de espiga, que aporta información complementaria y es más robusta frente a variaciones del umbral.

Ambas se agregan en ventanas de 20 ms, produciendo una secuencia a 50 Hz, y se normalizan mediante puntuación  $z$  dentro de cada bloque de registro [lightbeam]. Esta normalización por bloque es la primera línea de defensa frente a la no estacionariedad, pero, como se verá, resulta insuficiente por sí sola.

### 3.3.3. Organización de los datos

Los datos se distribuyen en ficheros HDF5 organizados por sesión de registro. Cada sesión se identifica por su fecha (`t15.AAAA.MM.DD`) y contiene hasta tres ficheros, correspondientes a las particiones de entrenamiento, validación y prueba. No todas las sesiones aportan a las tres particiones: algunas contienen únicamente datos de entrenamiento.

Dentro de cada fichero, cada ensayo se almacena como un grupo independiente con los siguientes elementos [b2t25]:

**input\_features.** Matriz de características neuronales de dimensiones  $T \times 512$ , donde  $T$  es el número de ventanas del ensayo.

**seq\_class\_ids.** Secuencia de identificadores de fonema correspondiente a la frase, es decir, la etiqueta fonémica de referencia.

**transcription.** Transcripción de la frase.

**Atributos.** Número de ventanas temporales, longitud de la secuencia de etiquetas, etiqueta textual de la frase, e identificadores de sesión, bloque y ensayo.

Es relevante para el diseño experimental que el conjunto **ya incluye la secuencia de fonemas de referencia**: el experimento de decodificación a nivel de fonema no requiere ningún sistema de conversión grafema-fonema ni diccionario de pronunciación adicional, lo que reduce sustancialmente su coste de preparación.

### 3.3.4. Particiones y composición por corpus

La cuadro 3.3 resume la composición del conjunto. El reparto es de 8.072 ensayos de entrenamiento, 1.426 de validación y 1.450 de prueba, estos últimos sin etiquetas, reservados para la evaluación oficial [lightbeam]. Los ensayos se agrupan en bloques de entre 5 y 50 frases; el conjunto de entrenamiento comprende 198 bloques repartidos por las 45 sesiones, mientras que validación y prueba suman 67 bloques procedentes de 41 de esas 45 sesiones.

**Tabla 3.3:** Composición del conjunto Brain-to-Text '25 por corpus de origen. Datos tomados de [lightbeam].

| Corpus de origen           | Bloques | Frases | Partición                         |
|----------------------------|---------|--------|-----------------------------------|
| Switchboard                | 182     | 7.672  | Entrenamiento y validación/prueba |
| Palabras frecuentes        | 48      | 2.050  | Entrenamiento y validación/prueba |
| Vocabulario de 50 palabras | 25      | 1.091  | Solo entrenamiento                |
| OpenWebText                | 5       | 172    | Solo validación/prueba            |
| Frases Harvard             | 3       | 98     | Solo validación/prueba            |
| Secuencias aleatorias      | 2       | 79     | Solo validación/prueba            |

Esta tabla merece atención porque revela un rasgo del *benchmark* que no es evidente a primera vista y que tiene consecuencias directas sobre la interpretación de los resultados. **Tres de los seis corpus de origen aparecen exclusivamente en validación y prueba, y nunca en entrenamiento:** OpenWebText, las frases Harvard y las secuencias aleatorias de palabras suman 349 frases que el modelo evalúa sin haber visto material lingüístico de esa procedencia. En sentido inverso, el vocabulario cerrado de 50 palabras aparece únicamente en entrenamiento.

Es decir, la partición de validación no es una muestra homogénea de la de entrenamiento: incorpora por diseño un **desplazamiento de dominio lingüístico**. Las secuencias aleatorias de palabras son, además, el caso adverso extremo para cualquier modelo de lenguaje, puesto que carecen por construcción de estructura sintáctica explorable. Esto explica parcialmente la marcada divergencia entre pérdida de entrenamiento y pérdida de validación que reportan los participantes de la competición, y obliga a matizar cualquier atribución de esa divergencia únicamente al sobreajuste del codificador.

### 3.3.5. Estrategias de habla no etiquetadas

Un último rasgo relevante: la mayor parte de los datos corresponde a intentos de habla *vocalizada*, pero una fracción corresponde a intentos de habla *silente*, y los organizadores decidieron deliberadamente **no proporcionar la etiqueta de estrategia** de cada bloque, con el objetivo de aproximar mejor las condiciones reales de uso [b2t25].

Se trata, por tanto, de una variable oculta que modula la señal y que el modelo debe absorber implícitamente. Ninguna de las variantes evaluadas en este trabajo la modela de forma explícita, lo que constituye una limitación reconocida y, a la vez, una línea de mejora identificable.

## 3.4. Análisis exploratorio

### 3.4.1. Distribución de longitudes

### 3.4.2. Variabilidad entre sesiones

### 3.4.3. Cobertura léxica y ejemplos por clase

### 3.4.4. Distribución temporal del registro

## 3.5. Retos que plantea el conjunto y consecuencias de diseño

Del análisis anterior se derivan cinco retos que condicionan directamente las decisiones tomadas en los capítulos siguientes:

- R1. No estacionariedad a lo largo de 20 meses.** Es el horizonte temporal más amplio de los *benchmarks* disponibles y multiplica por cinco el del conjunto anterior. La normalización por bloque no basta: motiva la incorporación de una transformación afín específica de cada sesión, y convierte la evaluación sobre sesiones no vistas en un experimento con valor propio.
- R2. Ausencia de alineamiento temporal.** Al no existir audio de referencia, no se dispone de correspondencia entre ventanas de señal y unidades emitidas. Determina el uso de CTC como objetivo de entrenamiento.
- R3. Escala de datos frente a escala de modelo.** Poco más de ocho mil frases de entrenamiento para un modelo de más de cien millones de parámetros sitúan el problema en un régimen de alto riesgo de sobreajuste, y hacen inviables las unidades de salida con vocabularios grandes: repartir ese número de frases entre decenas de miles de clases de subpalabra deja la mayoría de las clases con un puñado de ejemplos o ninguno.
- R4. Desplazamiento de dominio lingüístico entre particiones.** Los corpus presentes solo en validación penalizan a los sistemas que dependan en exceso del

modelo de lenguaje, y en particular las secuencias aleatorias de palabras actúan como caso adverso deliberado. Debe tenerse en cuenta al interpretar la diferencia entre las métricas de entrenamiento y validación.

**R5. Variable oculta de estrategia de habla.** La mezcla no etiquetada de intento vocalizado y silente introduce una fuente de variabilidad no modelada que se suma a la variabilidad entre sesiones.

Estos cinco puntos, junto con el marco de referencia establecido en el capítulo 2, definen el espacio de diseño del sistema que se describe a continuación.

# Capítulo 4

## Marco teórico

Este capítulo describe los fundamentos de los modelos y algoritmos empleados en el trabajo. A diferencia del capítulo 2, que revisa qué se ha hecho en el campo, aquí se detalla *cómo funciona* cada componente del sistema: la arquitectura del codificador preentrenado, el objetivo de entrenamiento que permite aprender sin alineamiento, la técnica de adaptación eficiente evaluada, el mecanismo de decodificación con modelo de lenguaje y las métricas con las que se cuantifican los resultados. La instanciación concreta de estos componentes en el sistema propuesto se aborda en el capítulo 5.

### 4.1. Planteamiento formal del problema

Sea  $\mathbf{X} = (\mathbf{x}_1, \dots, \mathbf{x}_T)$  una secuencia de  $T$  vectores de características neuronales, con  $\mathbf{x}_t \in \mathbb{R}^{512}$ , obtenida durante un intento de habla, y sea  $\mathbf{y} = (y_1, \dots, y_L)$  la secuencia de unidades lingüísticas correspondiente a la frase que el participante intentaba pronunciar, con  $y_l \in \mathcal{V}$  para un vocabulario  $\mathcal{V}$  de caracteres, fonemas, subpalabras o palabras.

El problema consiste en aprender una función  $f_\theta : \mathbb{R}^{T \times 512} \rightarrow \mathcal{V}^*$  que estime  $\mathbf{y}$  a partir de  $\mathbf{X}$ . Presenta tres características que determinan las decisiones metodológicas del trabajo:

- (a) **Es una tarea de transducción de secuencias de longitud variable**, con  $T \gg L$ : a 50 Hz, una frase de pocos segundos produce cientos de ventanas temporales frente a unas decenas de unidades de salida.
- (b) **No se dispone de alineamiento** entre las posiciones de  $\mathbf{X}$  y las de  $\mathbf{y}$ . Al no existir audio de referencia, no puede establecerse qué ventanas corresponden a qué unidad.

- (c) **El orden se preserva**, esto es, la correspondencia entre entrada y salida es monótona: a diferencia de la traducción automática, la unidad  $y_l$  nunca precede temporalmente a  $y_{l-1}$ .

La combinación de (b) y (c) es exactamente el escenario para el que se diseñó la clasificación temporal conexionista, descrita en la apartado 4.4.

## 4.2. Modelado de secuencias con codificadores atencionales

### 4.2.1. Autoatención multicabeza

El bloque fundamental de las arquitecturas empleadas es la autoatención por producto escalar escalado. Dada una secuencia de representaciones  $\mathbf{H} \in \mathbb{R}^{T \times d}$ , se proyecta en consultas, claves y valores mediante matrices aprendidas  $\mathbf{W}^Q, \mathbf{W}^K, \mathbf{W}^V$ , y se calcula

$$\text{Atencin}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V}, \quad (4.1)$$

donde el factor  $\sqrt{d_k}$  evita que el producto escalar crezca con la dimensión y sature la función *softmax*. En la variante multicabeza se realizan  $h$  proyecciones independientes en paralelo, cada una con dimensión  $d/h$ , y sus salidas se concatenan y se proyectan de nuevo, lo que permite que cada cabeza atienda a un tipo de relación distinto.

La propiedad relevante de este mecanismo es que su campo receptivo es **global**: cada posición puede atender a cualquier otra en una única capa, con coste cuadrático  $\mathcal{O}(T^2)$  en tiempo y memoria.

### 4.2.2. El compromiso entre contexto local y global en señales temporales

La autoatención pura resulta subóptima para señales de habla. Los fenómenos relevantes operan a dos escalas muy distintas: la estructura articulatoria es fundamentalmente **local** —transiciones entre fonemas contiguos, coarticulación—, mientras que la prosodia y la coherencia de la frase son **globales**. Un mecanismo puramente global debe aprender la localidad desde los datos, lo que resulta ineficiente en muestra.

La respuesta del campo ha sido combinar atención y convolución. El **Conformer** las encadena en serie dentro de cada bloque; el **Branchformer** las dispone en dos ramas paralelas cuyas salidas se combinan, lo que aporta interpretabilidad y flexibilidad; y

el **E-Branchformer** [kim2023branchformer], que es el codificador empleado por el modelo de este trabajo, mejora la etapa de combinación de esas dos ramas.

### 4.2.3. E-Branchformer

Cada bloque E-Branchformer procesa la entrada por dos ramas paralelas:

**Rama global.** Autoatención multicabeza según la ecuación (4.1), que modela dependencias de largo alcance.

**Rama local.** Un cgMLP (*convolutional gating MLP*), un perceptrón con una puerta convolucional de profundidad que captura patrones locales con un coste muy inferior al de la atención.

La aportación específica de E-Branchformer frente a Branchformer está en el **módulo de combinación**: en lugar de sumar o promediar las salidas de ambas ramas, las concatena y aplica una convolución de profundidad antes de la proyección final, lo que permite que la combinación dependa del contexto temporal en lugar de ser un promedio fijo. Los bloques incorporan además redes *feed-forward* en disposición macarrón, es decir, media red antes y media después del bloque de ramas.

En la configuración de referencia, la dimensión de las cabezas de atención es  $d/64$  y las dimensiones intermedias del cgMLP y de la red *feed-forward* son  $6d$  y  $4d$  respectivamente [kim2023branchformer]. Los bloques se apilan sobre un módulo de submuestreo convolucional que reduce la resolución temporal por un factor de cuatro, lo que mitiga el coste cuadrático de la atención.

## 4.3. OWSM v3.1

### 4.3.1. Origen y motivación

OWSM (*Open Whisper-style Speech Model*) reproduce el estilo de entrenamiento de Whisper [radford2023whisper] empleando exclusivamente datos públicos y la herramienta de código abierto ESPnet [watanabe2018espnet], publicando datos, recetas, registros de entrenamiento y pesos [peng2023owsm]. La versión v3.1 [peng2024owsmv31] sustituye el codificador Transformer de la versión anterior por E-Branchformer y se entrena sobre 180 000 h de habla pública, mejorando simultáneamente precisión y velocidad de inferencia.

Esta reproducibilidad es la razón por la que se escoge OWSM y no Whisper: dado que el trabajo pretende cuantificar *qué transfiere del preentrenamiento*, resulta meto-



dológicamente preferible partir de un modelo cuyos datos y procedimiento de entrenamiento son conocidos y auditables.

### 4.3.2. Arquitectura

OWSM v3.1 es un modelo codificador-decodificador compuesto por cuatro etapas:

1. **Frontend acústico**, que transforma la forma de onda en un banco de filtros logarítmico en escala mel.
2. **Módulo de submuestreo convolucional**, que reduce la resolución temporal y proyecta a la dimensión del codificador.
3. **Codificador E-Branchformer**, que produce una secuencia de representaciones contextualizadas.
4. **Decodificador Transformer autorregresivo**, que genera la transcripción sobre un vocabulario de subpalabras.

El modelo se publica en tres escalas: base con 101 millones de parámetros, *small* con 367 millones y *medium* con 1,02 mil millones [peng2024owsmv31].

### 4.3.3. Objetivo conjunto CTC-atención

Durante el preentrenamiento, OWSM se optimiza con una pérdida híbrida que combina la rama CTC sobre la salida del codificador con la entropía cruzada del decodificador autorregresivo [kim2017jointctc]:

$$\mathcal{L} = \lambda \mathcal{L}_{\text{CTC}} + (1 - \lambda) \mathcal{L}_{\text{att}}, \quad \lambda \in [0, 1]. \quad (4.2)$$

La motivación original de este esquema es que CTC impone una correspondencia monótona entre entrada y salida, lo que estabiliza el aprendizaje del alineamiento del decodificador atencional, que por sí solo es demasiado flexible y converge con dificultad.

Para este trabajo, la consecuencia práctica es determinante: al contemplar la arquitectura una rama CTC sobre la salida del codificador, **el modelo puede operarse en régimen puramente CTC** fijando  $\lambda = 1$  y prescindiendo por completo del decodificador autorregresivo. Lo que se reutiliza es la vía arquitectónica, no los pesos de la cabeza CTC preentrenada, que se sustituyen en cualquier caso al cambiar el vocabulario de salida. Esto elimina la dependencia del vocabulario de subpalabras preentrenado y del comportamiento generativo del decodificador, dos elementos que en un régimen de datos escasos resultan más un lastre que una ventaja.

## 4.4. Clasificación temporal conexionista (CTC)

### 4.4.1. Formulación

CTC [graves2006ctc] resuelve el problema de entrenar con pares  $(\mathbf{X}, \mathbf{y})$  sin alineamiento conocido. Introduce un símbolo adicional en blanco,  $\emptyset$ , ampliando el vocabulario a  $\mathcal{V}' = \mathcal{V} \cup \{\emptyset\}$ , y define un **alineamiento**  $\boldsymbol{\pi} \in \mathcal{V}'^T$  como una asignación de un símbolo a cada una de las  $T$  posiciones de la secuencia de entrada.

La función de colapso  $\mathcal{B} : \mathcal{V}'^T \rightarrow \mathcal{V}^*$  transforma un alineamiento en una etiqueta eliminando primero las repeticiones consecutivas y después los blancos. Así, tanto  $(h, h, \emptyset, o, o)$  como  $(\emptyset, h, \emptyset, \emptyset, o)$  colapsan a  $ho$ . El blanco es necesario precisamente para poder representar símbolos repetidos legítimos:  $oo$  exige un blanco intercalado.

Asumiendo independencia condicional entre posiciones dada la entrada, la probabilidad de un alineamiento es el producto de las probabilidades por posición, y la probabilidad de una etiqueta es la suma sobre todos los alineamientos que colapsan a ella:

$$p(\boldsymbol{\pi} \mid \mathbf{X}) = \prod_{t=1}^T p(\pi_t \mid \mathbf{X}), \quad (4.3)$$

$$p(\mathbf{y} \mid \mathbf{X}) = \sum_{\boldsymbol{\pi} \in \mathcal{B}^{-1}(\mathbf{y})} p(\boldsymbol{\pi} \mid \mathbf{X}). \quad (4.4)$$

La pérdida es la log-verosimilitud negativa,  $\mathcal{L}_{\text{CTC}} = -\log p(\mathbf{y} \mid \mathbf{X})$ . El número de alineamientos crece exponencialmente con  $T$ , pero la suma de la ecuación (4.4) se calcula de forma exacta en tiempo  $\mathcal{O}(T \cdot L)$  mediante un algoritmo de programación dinámica hacia delante y hacia atrás, análogo al empleado en modelos ocultos de Markov.

### 4.4.2. Decodificación voraz

La decodificación más simple toma el símbolo más probable en cada posición y aplica la función de colapso:

$$\hat{\mathbf{y}}_{\text{voraz}} = \mathcal{B} \left( \arg \max_{\pi_t} p(\pi_t \mid \mathbf{X}) \forall t \right). \quad (4.5)$$

Es una aproximación al máximo a posteriori, no su solución exacta, porque la etiqueta más probable puede no corresponder al alineamiento más probable. Su interés en este trabajo es que **aisla la calidad del codificador** de la del modelo de lenguaje:

es la métrica adecuada para comparar codificadores entre sí.

#### 4.4.3. Regularización intermedia (InterCTC)

Una técnica habitual en codificadores profundos consiste en aplicar una pérdida CTC auxiliar sobre la salida de una capa intermedia, además de la pérdida principal sobre la última capa:

$$\mathcal{L} = (1 - \alpha_{\text{ic}}) \mathcal{L}_{\text{CTC}}^{(N)} + \alpha_{\text{ic}} \mathcal{L}_{\text{CTC}}^{(k)}, \quad k < N. \quad (4.6)$$

El efecto es doble: proporciona una señal de gradiente más directa a las capas inferiores, mitigando problemas de propagación en redes profundas, y actúa como regularizador al forzar que las representaciones intermedias sean ya linealmente decodificables.

#### 4.4.4. Propiedades y limitaciones

La hipótesis de independencia condicional de la ecuación (4.3) es la clave de la tratabilidad del método y, a la vez, su principal debilidad: el modelo no condiciona la predicción de una posición a lo emitido en las anteriores, por lo que **no incorpora ningún modelo de lenguaje implícito**. Esto explica por qué la salida CTC voraz de un sistema *brain-to-text* produce cadenas fonéticamente plausibles pero ortográficamente incorrectas, y por qué la etapa de decodificación con modelo de lenguaje externo resulta imprescindible.

Conviene señalar una consecuencia práctica que afecta a la interpretación de los resultados: **la pérdida CTC y la tasa de error no siempre se mueven en la misma dirección**. La pérdida mide la masa de probabilidad asignada al conjunto de alineamientos correctos, mientras que la tasa de error mide el acierto de la decodificación voraz. Un modelo puede volverse más confiado en alineamientos correctos sin cambiar su predicción de máximo, o al revés. Por ello, la selección de modelo debe realizarse sobre la métrica de error y no sobre la pérdida de validación.

### 4.5. Adaptación eficiente en parámetros: LoRA

LoRA [hu2022lora] parte de la hipótesis de que la actualización de pesos necesaria para adaptar un modelo preentrenado a una tarea nueva tiene rango intrínseco bajo. Dada una matriz de pesos preentrenada  $\mathbf{W}_0 \in \mathbb{R}^{d \times k}$ , en lugar de aprender una actualización densa  $\Delta \mathbf{W}$  se la factoriza como producto de dos matrices de rango  $r \ll \min(d, k)$ :

$$\mathbf{W} = \mathbf{W}_0 + \Delta\mathbf{W} = \mathbf{W}_0 + \frac{\alpha}{r} \mathbf{B}\mathbf{A}, \quad \mathbf{B} \in \mathbb{R}^{d \times r}, \mathbf{A} \in \mathbb{R}^{r \times k}. \quad (4.7)$$

Durante el entrenamiento  $\mathbf{W}_0$  permanece congelada y sólo se actualizan  $\mathbf{A}$  y  $\mathbf{B}$ , lo que reduce el número de parámetros entrenables de  $d \times k$  a  $r(d + k)$ . La matriz  $\mathbf{A}$  se inicializa de forma aleatoria y  $\mathbf{B}$  a cero, de modo que  $\Delta\mathbf{W} = \mathbf{0}$  al inicio y el modelo parte exactamente del comportamiento preentrenado. El factor  $\alpha/r$  desacopla la magnitud efectiva de la actualización de la elección del rango.

Es esencial subrayar la condición de aplicabilidad: **LoRA presupone que  $\mathbf{W}_0$  está congelada**. Su beneficio no procede de añadir capacidad, sino de *restringir* la adaptación a un subespacio de rango bajo, lo que actúa como regularización y reduce el coste. Aplicado sobre un modelo cuyos pesos base también se actualizan, la restricción desaparece: la suma  $\mathbf{W}_0 + \frac{\alpha}{r} \mathbf{B}\mathbf{A}$  con ambos términos entrenables es simplemente una reparametrización redundante de una matriz densa entrenable. Cualquier evaluación experimental de LoRA debe, por tanto, controlar explícitamente el estado de congelación del modelo base.

## 4.6. Modelado de lenguaje y decodificación

### 4.6.1. Modelos de lenguaje de $n$ -gramas

Un modelo de lenguaje asigna una probabilidad a una secuencia de palabras. Bajo la aproximación de Markov de orden  $n - 1$ , la probabilidad de cada palabra depende únicamente de las  $n - 1$  anteriores:

$$P(w_1, \dots, w_M) \approx \prod_{i=1}^M P(w_i \mid w_{i-n+1}, \dots, w_{i-1}). \quad (4.8)$$

Las probabilidades se estiman por máxima verosimilitud sobre un corpus y se corrigen mediante técnicas de suavizado y retroceso, que reasignan masa de probabilidad a los  $n$ -gramas no observados. Sin suavizado, cualquier secuencia que contenga un  $n$ -grama ausente del corpus recibiría probabilidad nula.

La elección del orden  $n$  es un compromiso: un orden mayor captura más contexto, pero el número de  $n$ -gramas posibles crece exponencialmente y, con un corpus limitado, la mayoría queda sin observar, de modo que el modelo se apoya cada vez más en el retroceso y su ventaja se desvanece. Con corpus del orden de unos pocos miles de frases, órdenes bajos resultan preferibles. Para la consulta eficiente de estos modelos se emplea KenLM [heafield2011kenlm], que utiliza estructuras de datos optimizadas en memoria

y tiempo de acceso.

#### 4.6.2. Búsqueda en haz sobre salidas CTC

La decodificación voraz de la ecuación (4.5) no permite incorporar información lingüística. La alternativa es una búsqueda en haz que mantiene, en cada instante  $t$ , un conjunto acotado de las  $k$  hipótesis parciales más prometedoras, extendiéndolas símbolo a símbolo. La particularidad del caso CTC es que hipótesis distintas pueden colapsar a la misma etiqueta, por lo que la búsqueda debe operar sobre etiquetas colapsadas y acumular la probabilidad de todos los alineamientos que conducen a cada una.

#### 4.6.3. Fusión superficial

La integración del modelo de lenguaje se realiza mediante fusión superficial, sumando su contribución a la puntuación de cada hipótesis durante la búsqueda:

$$s(\mathbf{y}) = \log p_{\text{CTC}}(\mathbf{y} \mid \mathbf{X}) + \alpha \log P_{\text{LM}}(\mathbf{y}) + \beta |\mathbf{y}|_w, \quad (4.9)$$

donde  $\alpha$  pondera la influencia del modelo de lenguaje y  $\beta$  es una bonificación por inserción de palabra que compensa el sesgo del término lingüístico hacia hipótesis cortas, ya que cada palabra adicional multiplica por una probabilidad menor que uno. Ambos hiperparámetros se ajustan empíricamente sobre la partición de validación.

#### 4.6.4. Guiado frente a reordenamiento, y el límite *oracle*

Existen dos regímenes cualitativamente distintos de uso del modelo de lenguaje, y distinguirlos es una de las preguntas de investigación de este trabajo:

**Reordenamiento.** El modelo acústico genera una lista de las  $n$  mejores hipótesis y el modelo de lenguaje se limita a reordenarla. Su rendimiento está acotado superiormente por la mejor hipótesis contenida en esa lista.

**Guiado.** El modelo de lenguaje interviene *durante* la búsqueda, según la ecuación (4.9), alterando qué hipótesis se expanden y cuáles se podan. Puede así alcanzar transcripciones que nunca habrían figurado en la lista  $n$ -mejor puramente acústica.

El **límite *oracle*** de la lista  $n$ -mejor —el error que se obtendría seleccionando siempre la mejor hipótesis disponible, usando la referencia para elegir— cuantifica ese techo. Es una cota que ningún método de reordenamiento puede superar. En consecuencia,

si un sistema con fusión superficial obtiene un error *inferior* al límite *oracle* de su propia lista  $n$ -mejor acústica, queda demostrado empíricamente que el modelo de lenguaje no está reordenando sino guiando la búsqueda, y que una parte del conocimiento lingüístico del sistema completo reside fuera del codificador neuronal.

## 4.7. Métricas de evaluación

### 4.7.1. Distancia de edición

Todas las métricas empleadas derivan de la distancia de Levenshtein entre la secuencia predicha y la de referencia, definida como el número mínimo de operaciones de sustitución ( $S$ ), eliminación ( $D$ ) e inserción ( $I$ ) necesarias para transformar una en otra. Normalizada por la longitud  $N$  de la referencia, produce una tasa de error:

$$\text{Tasa de error} = \frac{S + D + I}{N}. \quad (4.10)$$

Nótese que la tasa **no está acotada superiormente por la unidad**: una hipótesis mucho más larga que la referencia puede producir valores superiores al 100 % por acumulación de inserciones.

### 4.7.2. CER, PER y WER

Según la unidad sobre la que se aplique la ecuación (4.10) se obtienen tres métricas:

**CER** (*Character Error Rate*). Sobre caracteres. Es la métrica natural para evaluar un codificador entrenado con objetivo de carácter y decodificación voraz, ya que no depende de ningún componente lingüístico externo.

**PER** (*Phoneme Error Rate*). Sobre fonemas. Es la métrica de selección de modelo habitual en la literatura de neuroprótesis del habla, por ser el fonema la unidad de salida dominante.

**WER** (*Word Error Rate*). Sobre palabras del texto final. Es la métrica oficial de los *benchmarks* y la única con relevancia directa para el usuario, ya que mide el resultado que efectivamente aparecería en pantalla.

### 4.7.3. Comparabilidad entre métricas

Un punto metodológico que condiciona el diseño experimental: **CER, PER y WER no son comparables entre sí**. Se calculan sobre unidades de distinta gra-

nularidad y sobre denominadores distintos, de modo que un CER de 0,25 y un PER de 0,25 no representan sistemas equivalentes.

La consecuencia es que la comparación entre modelos entrenados con *unidades de salida distintas* sólo puede realizarse sobre el WER del texto final decodificado, que es el único eje común. Toda comparación de granularidad exige, por tanto, decodificar hasta texto, con las mismas condiciones de modelo de lenguaje para todas las variantes. Por el contrario, la comparación entre modelos que comparten unidad de salida puede y debe hacerse sobre la métrica correspondiente con decodificación voraz, que aísla mejor la contribución del codificador.

# Capítulo 5

## Arquitectura del sistema

Este capítulo describe el sistema propuesto: qué se conserva del modelo preentrenado, qué se sustituye, y qué componentes se añaden desde cero. Mientras que el capítulo 4 presenta los mecanismos de forma genérica, aquí se detalla su instanciación concreta, incluyendo las decisiones de implementación que no son evidentes a partir de la descripción teórica y que condicionaron el resultado.

### 5.1. Visión general

El sistema toma como punto de partida OWSM v3.1 en su variante *base* sobre codificador E-Branchformer (`espnet/owsm_v3.1_ebf_base`, 101 millones de parámetros) y lo transforma en un decodificador de actividad neuronal mediante cuatro intervenciones:

1. **Eliminación de la cadena acústica.** El *frontend* de banco de filtros mel y la normalización global se desactivan por completo: la señal neuronal entra directamente al modelo.
2. **Sustitución del módulo de entrada** del codificador por un adaptador entrenado desde cero, que proyecta las 512 características neuronales al espacio de 384 dimensiones que espera el codificador y realiza el submuestreo temporal.
3. **Inserción de una capa específica de sesión** delante del adaptador, que compensa la deriva de la señal entre días de registro.
4. **Sustitución de la cabeza CTC** por una nueva, dimensionada según la unidad lingüística de salida elegida, y entrenada desde cero.

El codificador E-Branchformer, que constituye el grueso de los parámetros, se conserva estructuralmente intacto; lo que varía entre experimentos es su inicialización y



su régimen de congelación. La figura 5.1 resume el sistema completo.

[FIGURA: diagrama de bloques del sistema completo. De izquierda a derecha: tensor de entrada ( $T \times 512$ )  $\rightarrow$  normalización y aumento  $\rightarrow$  capa de sesión (con el canal 513 de índice de sesión)  $\rightarrow$  módulo adaptador (submuestreo  $\times 4$ , salida 384)  $\rightarrow$  codificación posicional  $\rightarrow$  6 bloques E-Branchformer  $\rightarrow$  cabeza CTC  $\rightarrow$  decodificación. Marcar en un color los bloques preentrenados y en otro los entrenados desde cero.]

**Figura 5.1:** Arquitectura general del sistema propuesto.

## 5.2. Adaptación de modalidad

### 5.2.1. Qué se conserva y qué se sustituye

La cuadro 5.1 detalla el tratamiento de cada componente del modelo original.

**Tabla 5.1:** Tratamiento de cada componente de OWSM v3.1 en el sistema propuesto.

| Componente original          | Tratamiento        | Motivo                           |
|------------------------------|--------------------|----------------------------------|
| Frontend mel                 | Eliminado          | La entrada no es audio           |
| Normalización global         | Eliminada          | Los datos ya vienen normalizados |
| Submuestreo convolucional    | Sustituido         | Dimensión de entrada distinta    |
| Codificación posicional      | <b>Reutilizada</b> | Independiente de la modalidad    |
| Codificador E-Branchformer   | Conservado         | Es el objeto del estudio         |
| Decodificador autorregresivo | Desactivable       | Innecesario en régimen CTC puro  |
| Cabeza CTC                   | Sustituida         | Vocabulario de salida distinto   |

Merece un comentario la reutilización de la codificación posicional. El módulo original de submuestreo de OWSM contiene, además de las convoluciones, la codificación posicional sinusoidal que el codificador espera recibir. Al sustituir el módulo completo se perdería también esa codificación, de modo que el sistema la extrae del modelo

original antes de reemplazarlo y la reinyecta en el nuevo adaptador. Es una operación conservadora: la codificación posicional no depende de la modalidad de entrada, sólo de la posición temporal, por lo que no hay motivo para reaprenderla.

### 5.2.2. Desactivación del decodificador

Cuando el peso de la rama CTC se fija a la unidad, ESPnet desactiva automáticamente el decodificador autorregresivo. Esto tiene dos consecuencias que conviene tener presentes al leer los registros de entrenamiento:

- El sistema opera en régimen CTC puro y la única decodificación posible es voraz o por búsqueda en haz sobre la salida CTC, no generativa.
- La métrica `valid_cer` que ESPnet reporta por defecto corresponde al decodificador de atención y **deja de existir**. La métrica válida pasa a ser `valid_cer_ctc`. Seleccionar modelo por la métrica equivocada, o por la pérdida total, produce elecciones incorrectas: con peso de CTC bajo, la pérdida total está dominada por el término de atención, que toca fondo en pocas épocas gracias al prior del inglés preentrenado y no refleja la calidad de la decodificación neuronal.

## 5.3. Capa específica de sesión

### 5.3.1. Motivación y formulación

Como se estableció en el capítulo 3, las 45 sesiones abarcan 20 meses y la señal asociada a un mismo fonema varía entre días por deriva y micromovimiento de los electrodos. Un adaptador único compartido tendría que aprender simultáneamente múltiples correspondencias contradictorias entre señal y unidad lingüística, además de inferir implícitamente a qué día pertenece cada ensayo.

La solución adoptada sigue la práctica estándar del campo, introducida en [willett2021] y presente en la mayoría de los sistemas posteriores (apartado 2.3): aprender una transformación de entrada propia de cada día. La aportación de este trabajo no está en la idea, sino en su instanciación concreta y en su evaluación experimental, incluida la cuantificación de su coste y de su comportamiento ante días no vistos.

La capa de sesión aplica una transformación afín distinta por cada día de registro, seguida de una no linealidad acotada:

$$\mathbf{z}_t = \text{softsign}(\mathbf{W}_s \mathbf{x}_t + \mathbf{b}_s), \quad \mathbf{W}_s \in \mathbb{R}^{512 \times 512}, \mathbf{b}_s \in \mathbb{R}^{512}, \quad (5.1)$$

donde  $s$  es el índice de la sesión a la que pertenece el ensayo y  $\text{softsign}(u) = u/(1 + |u|)$  es una no linealidad acotada en  $(-1, 1)$ , de saturación más suave que la tangente hiperbólica, que impide que una matriz de sesión mal condicionada propague valores extremos al adaptador. Cada matriz  $\mathbf{W}_s$  se inicializa a la identidad y cada vector  $\mathbf{b}_s$  a cero, de modo que al comenzar el entrenamiento la capa se limita a aplicar la función *softsign* y el sistema parte de un comportamiento neutro.

El coste en parámetros es de  $n_{\text{ses}} \times (512^2 + 512)$ , es decir, unos 2,6 millones con 10 sesiones y unos 12 millones con las 45. Es una cantidad considerable, comparable a una fracción apreciable del propio codificador, y su beneficio debe por tanto justificarse experimentalmente y no asumirse.

*[FIGURA: esquema de la capa de sesión. Mostrar el banco de  $n_{\text{ses}}$  matrices  $\mathbf{W}_s$ , la selección por índice de sesión, y cómo días distintos se proyectan a un espacio común antes del adaptador. Útil superponer un esquema de la deriva entre sesiones.]*

**Figura 5.2:** Transformación afín específica de sesión.

### 5.3.2. Transporte del índice de sesión

La capa necesita saber a qué sesión pertenece cada ensayo, pero la cadena de datos de ESPnet transporta únicamente el tensor de señal. La solución adoptada consiste en **añadir el índice de sesión como un canal adicional constante en el tiempo**, de modo que la entrada pasa de  $T \times 512$  a  $T \times 513$ . La capa de sesión lee ese índice del primer instante temporal, lo separa del resto y aplica la transformación correspondiente.

La elección del primer instante no es arbitraria: el relleno de las secuencias hasta la longitud del lote se añade *al final* y con ceros, por lo que el instante inicial corresponde siempre a datos reales. Leer el índice del último instante devolvería cero para cualquier ensayo más corto que el más largo del lote.

## 5.4. Módulo adaptador

El adaptador sustituye al módulo de submuestreo original y cumple tres funciones: mezclar la información entre electrodos, reducir la resolución temporal y proyectar a la dimensión del codificador. Se implementaron cuatro variantes de complejidad creciente para poder cuantificar experimentalmente el compromiso entre capacidad y coste.

### 5.4.1. Variante convolucional bidimensional

Reutiliza el módulo original de OWSM, un submuestreo convolucional en dos dimensiones que trata la entrada como una imagen de tiempo por frecuencia. Es la opción de menor esfuerzo de implementación, pero conceptualmente cuestionable: convoluciona a lo largo del eje de electrodos como si fuera un eje de frecuencias, cuando en realidad **el orden de los electrodos es arbitrario** y no existe ninguna noción de proximidad entre índices consecutivos. Es además con diferencia la más costosa, del orden de 66 GFLOP por muestra, lo que supone entre el 85 % y el 90 % del coste total del modelo.

### 5.4.2. Variante lineal

Una única proyección lineal de 512 a 384 dimensiones seguida de una decimación temporal por un factor de cuatro. Es la variante de menor capacidad y sirve como cota inferior de referencia: cualquier adaptador más complejo debe justificar su coste frente a ella.

### 5.4.3. Variante convolucional unidimensional

Dos convoluciones temporales de núcleo tres y paso dos con activaciones rectificadas, que realizan simultáneamente la mezcla y el submuestreo por un factor de cuatro. Es un punto intermedio razonable en coste.

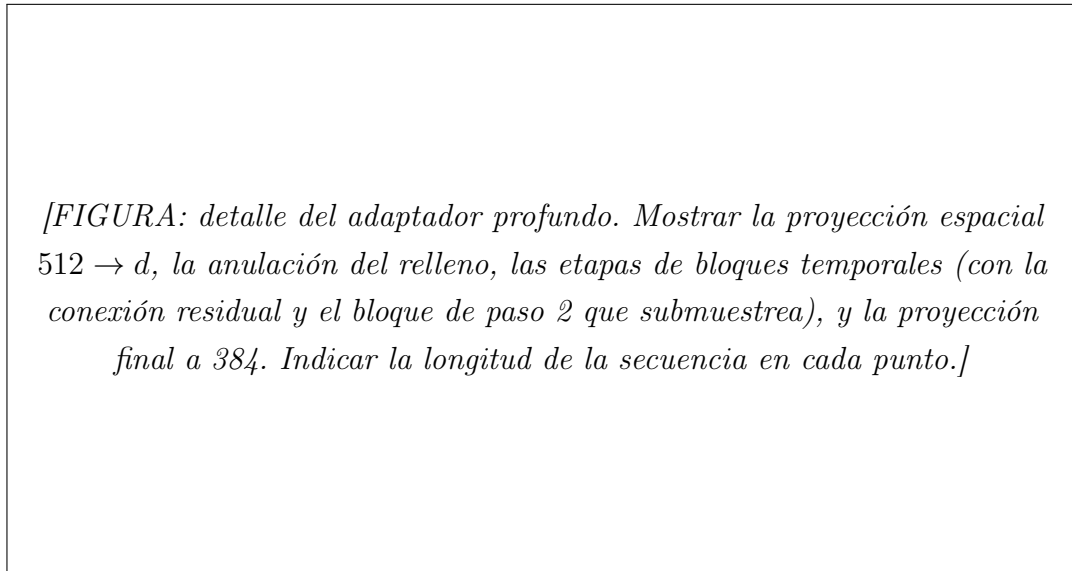
### 5.4.4. Variante profunda

Es la variante diseñada específicamente para señal neuronal, y separa de forma explícita las dos operaciones que la variante bidimensional confunde:

1. **Proyección espacial.** Una capa lineal de 512 a  $d$  dimensiones, seguida de normalización por capas, activación GELU y descarte aleatorio. Mezcla *todos los electrodos con todos*, que es la operación correcta cuando el orden de los electrodos carece de significado, a diferencia de la convolución bidimensional.

2. **Bloques temporales.** Una pila de convoluciones unidimensionales de núcleo  $k$  con normalización por capas, activación GELU, descarte y conexión residual. El submuestreo se realiza mediante el paso de la convolución, no por decimación, de modo que **ninguna muestra se descarta sin haber sido vista**. La pila se organiza en etapas: cada etapa de submuestreo por dos va seguida de  $n_{\text{res}}$  bloques residuales de paso unitario.
3. **Proyección de salida.** Una capa lineal de  $d$  a 384 dimensiones y la codificación posicional heredada de OWSM.

Los valores de referencia son  $d = 512$ ,  $n_{\text{res}} = 2$  bloques residuales por etapa, núcleo  $k = 5$  y descarte de 0,1. El factor de submuestreo total es configurable entre uno y ocho; con ventanas de 20 ms, un factor de cuatro produce tramas de 80 ms y un factor de dos, tramas de 40 ms con el doble de coste.



**Figura 5.3:** Estructura del módulo adaptador profundo.

#### 5.4.5. Anulación del relleno

Un detalle de implementación con consecuencias reales: tras la proyección espacial, que incluye un término de sesgo, los instantes de relleno **dejan de valer cero**. Si no se corrigen, las convoluciones temporales posteriores mezclan ese contenido espurio con los datos reales de los instantes finales de cada secuencia. El adaptador profundo vuelve a anular explícitamente las posiciones de relleno tras la proyección espacial y antes de los bloques temporales.

## 5.5. Cabeza CTC y unidades de salida

La proyección lineal final del modelo original mapea la salida del codificador a las 50.002 clases del vocabulario de subpalabras de OWSM. El sistema la sustituye por una nueva proyección dimensionada según la unidad de salida elegida, entrenada desde cero. Se implementaron cinco objetivos:

**Tabla 5.2:** Unidades de salida CTC implementadas.

| Objetivo           | Clases   | Origen                                                          | Métrica |
|--------------------|----------|-----------------------------------------------------------------|---------|
| Subpalabra de OWSM | 50.002   | Vocabulario preentrenado                                        | CER/WER |
| Carácter           | 29       | 26 letras, apóstrofo, espacio, blanco                           | CER/WER |
| Fonema             | Variable | Etiquetas del propio HDF5                                       | PER     |
| Subpalabra propia  | 256      | <i>SentencePiece</i> sobre las transcripciones de entrenamiento | CER/WER |
| Palabra            | Variable | Vocabulario cerrado de entrenamiento                            | WER     |

Dos observaciones sobre este componente. La primera es que **esta capa es la que hace inviable el vocabulario de subpalabras preentrenado**: se entrena desde cero, y repartir poco más de ocho mil frases entre 50.002 clases deja a la inmensa mayoría sin ejemplos, mientras que con 29 clases de carácter cada una recibe miles.

La segunda es que el objetivo de fonema no produce texto ortográfico, por lo que compararlo con la transcripción de referencia carecería de sentido. En esa configuración el sistema reporta tasa de error de fonema y deja las métricas de texto sin valor, ya que obtenerlas exigiría un decodificador con léxico. Esto es precisamente lo que motiva que la comparación entre granularidades deba hacerse sobre el texto final decodificado, según se argumentó en la apartado 4.7.

Adicionalmente, el calculador de errores interno de ESPnet debe recibir la lista de símbolos del objetivo activo. De no hacerlo, las cifras que aparecen en el registro de entrenamiento se calculan con el vocabulario equivocado y **carecen de significado**, sin que nada en el registro lo advierta.

## 5.6. Estrategias de adaptación del codificador

El sistema congela inicialmente la totalidad de los parámetros y descongela después de forma selectiva. El adaptador y la cabeza CTC, al ser componentes nuevos, están siempre activos. Sobre el codificador se contemplan las siguientes estrategias:

**Codificador congelado.** Sólo se entrenan el adaptador y la cabeza CTC. Mide cuánto puede extraerse de las representaciones preentrenadas sin modificarlas.

**Descongelado total.** Todos los parámetros del codificador se actualizan.

**Descongelado parcial.** Se descongelan únicamente las  $N$  capas superiores, las más próximas a la salida y más específicas de la tarea, junto con la normalización final del codificador.

**LoRA.** Adaptadores de rango bajo sobre las proyecciones de atención y de las redes *feed-forward* del codificador.

### 5.6.1. Interacción entre LoRA y la congelación

Conforme al argumento formal de la apartado 4.5, el sistema **desactiva LoRA automáticamente** cuando se solicita junto con el codificador completamente descongelado, y emite un aviso. En esa combinación los adaptadores de rango bajo no aportan restricción alguna y sólo añaden parámetros y ciclos de fusión. Dejar que esa configuración se ejecutase habría producido filas en la tabla de resultados que aparentan evaluar LoRA sin hacerlo.

### 5.6.2. Tasa de aprendizaje diferencial

El sistema permite escalar la tasa de aprendizaje del codificador por un factor, manteniendo la tasa base para el adaptador, la cabeza CTC y los adaptadores de rango bajo. La separación se realiza por nombre de parámetro al construir el optimizador.

La motivación inicial fue la habitual: aplicar una misma tasa a componentes preentrenados y a componentes inicializados desde cero es cuestionable, porque los primeros parten de un óptimo y los segundos del ruido. Sin embargo, el mecanismo resultó ser necesario también **con el codificador inicializado aleatoriamente**, donde ese argumento no aplica: con tasa unificada el entrenamiento diverge. La tasa diferenciada cumple aquí, por tanto, una segunda función independiente de la procedencia de los pesos, la de estabilizar el acoplamiento entre un adaptador que aprende deprisa y un codificador profundo que aún no ha organizado sus representaciones. Ambas funciones se distinguen explícitamente al interpretar los resultados.

### 5.6.3. Codificador recortado

Una última capacidad, motivada por los resultados del control de inicialización: cuando el codificador se inicializa aleatoriamente, deja de existir la restricción de que

su estructura coincida con la del modelo preentrenado, y puede reducirse su número de bloques. Esto permite formular la pregunta natural que sigue al control científico: si los pesos preentrenados no aportan, ¿aporta al menos el tamaño de la arquitectura? La configuración es incompatible con la inicialización preentrenada y el sistema lo verifica explícitamente.

#### 5.6.4. El control de inicialización

El interruptor más importante del sistema no añade ningún componente: permite **omitir la carga de los pesos preentrenados**, dejando el codificador con inicialización aleatoria y manteniendo absolutamente todo lo demás idéntico. Es el control científico sobre el que descansa la primera pregunta de investigación: sin este punto de comparación, cualquier afirmación sobre la aportación del preentrenamiento sería una suposición y no una medida.

### 5.7. Preprocesado y aumento de datos

#### 5.7.1. Normalización

Se contemplan dos regímenes: no aplicar ninguna normalización adicional, dado que los datos distribuidos ya vienen normalizados por bloque de registro y así los emplean los trabajos publicados, o aplicar una puntuación  $z$  adicional por ensayo y canal. Esta segunda opción tiene un efecto que conviene explicitar: al normalizar cada ensayo por separado, **se elimina la amplitud relativa entre ensayos**, que podría ser informativa.

#### 5.7.2. Aumento de datos

Se implementaron cuatro transformaciones, aplicadas únicamente durante el entrenamiento:

**Ruido blanco gaussiano.** Sumado a la señal ya normalizada. Es el regularizador principal del sistema de referencia del *benchmark*, no un detalle accesorio.

**Desplazamiento constante por canal.** Un valor por electrodo, sorteado una vez por ensayo, que imita la deriva de línea base *dentro* de una misma sesión, que la capa de sesión no puede corregir por variar de un ensayo a otro.

**Bandas temporales anuladas.** Intervalos contiguos de tiempo puestos a cero.



**Electrodos anulados.** Un subconjunto **aleatorio, no contiguo**, de electrodos puesto a cero. La no contigüidad es deliberada y coherente con el argumento de la apartado 5.4.4: al ser arbitrario el orden de los electrodos, anular un intervalo de índices consecutivos no tiene el significado físico que sí tiene anular una banda de frecuencias en audio.

### 5.7.3. Suavizado temporal

El suavizado gaussiano a lo largo del eje temporal **no es aumento de datos sino preprocesado**, y por tanto se aplica por igual en entrenamiento, validación e inferencia. Se aplica después del ruido, de modo que el suavizado tenga efecto sobre él, siguiendo la práctica del sistema de referencia. Confundir ambas categorías conduciría a un desajuste entre las condiciones de entrenamiento y las de evaluación.

## 5.8. Recuento de parámetros

**Tabla 5.3:** Distribución de parámetros del sistema en la configuración de referencia.

| Componente                             | Parámetros | % | Estado           |
|----------------------------------------|------------|---|------------------|
| Codificador E-Branchformer (6 bloques) | —          | — | Según estrategia |
| Adaptador (variante profunda)          | —          | — | Entrenable       |
| Capa de sesión                         | —          | — | Entrenable       |
| Cabeza CTC                             | —          | — | Entrenable       |
| Decodificador autorregresivo           | —          | — | Desactivado      |
| <b>Total entrenable</b>                | —          | — |                  |
| <b>Total del modelo</b>                | —          | — |                  |

## 5.9. Decisiones de implementación

Varias restricciones de la herramienta condicionaron el diseño y merecen quedar documentadas, tanto por transparencia como por su utilidad para quien reproduzca el trabajo.

**Herencia obligada del módulo de submuestreo.** El codificador de ESPnet decide si pasar la máscara de longitudes al módulo de entrada comprobando su tipo. Los adaptadores deben, por tanto, heredar de la clase de submuestreo original aunque no utilicen su constructor; en caso contrario el codificador los invoca sin

máscara, asumiendo que se trata de una incrustación de texto que no submuestra, y la ejecución falla.

**Propagación de la máscara.** Cada adaptador debe recortar o rellenar la máscara de longitudes para que su tamaño coincida exactamente con la secuencia submuestreada, ya que las convoluciones con relleno no siempre producen la longitud que la división entera predice.

**Inyección del peso de la rama CTC.** El peso de la rama CTC debe inyectarse en la configuración del modelo y no en la del entrenamiento. Construido desde la configuración de preentrenamiento, el modelo conservaría el valor original de OWSM con independencia de lo indicado, y la pérdida optimizada no sería la esperada.

**Lectura perezosa del almacenamiento.** Con las 45 sesiones, los datos superan la memoria disponible, de modo que el conjunto almacena únicamente un índice de pares fichero-clave y lee cada ensayo bajo demanda. Los descriptors de fichero deben cerrarse antes de cualquier bifurcación de proceso, ya que la biblioteca de acceso a HDF5 no es segura frente a ello.

**Verificación de integridad de las copias.** La copia de los datos desde el almacenamiento en la nube al disco local puede interrumpirse a medias, dejando un fichero truncado que las comprobaciones de existencia dan por válido y que falla a mitad de entrenamiento con un error opaco. El sistema compara los tamaños de origen y destino y vuelve a copiar cuando no coinciden.

**Trazabilidad de los experimentos.** Cada experimento se identifica por una huella derivada de su configuración completa, lo que permite detectar experimentos ya ejecutados, reintentar los que fallaron y garantizar que dos filas de la tabla de resultados con la misma etiqueta corresponden efectivamente a la misma configuración.

# Capítulo 6

## Metodología experimental

Este capítulo describe cómo se ejecutan, miden y comparan los experimentos. Su objetivo es que cualquier cifra del capítulo 7 pueda interpretarse sabiendo exactamente bajo qué condiciones se obtuvo y qué diferencias entre configuraciones son atribuibles a la configuración y cuáles al ruido de medida.

### 6.1. Entorno de ejecución y presupuesto de cómputo

Todo el trabajo se ejecuta en Google Colab sobre una única GPU NVIDIA L4, empleando ESPnet [watanabe2018espnet] a través de su interfaz ESPnet-EZ [someki2024espnetez], que permite construir y entrenar modelos desde Python sin recurrir a las recetas de línea de órdenes.

Los datos residen en almacenamiento en la nube, pero **no se entrena leyendo directamente de él**: el servicio limita las operaciones de entrada y salida por fichero, y con lectura perezosa sobre 45 sesiones eso se convierte en el cuello de botella. Cada sesión se copia una vez al disco local de la máquina virtual, con verificación de integridad por tamaño, y el entrenamiento lee de ahí. Los puntos de control se escriben también en local y sólo el mejor se copia de vuelta al almacenamiento persistente, junto con los registros y las métricas.

El coste se contabiliza en unidades de cómputo del servicio, que es la restricción efectiva del proyecto. A partir de mediciones reales sobre ejecuciones completas se calibró un estimador del coste por época en función de la configuración, que permite planificar cada bloque de experimentos antes de lanzarlo y decidir qué recortar cuando el presupuesto aprieta. Con 45 sesiones, el coste de referencia es del orden de ocho minutos por época.

## 6.2. Preparación de los datos

Los conjuntos de entrenamiento y validación se construyen recorriendo las claves de cada fichero HDF5 sin leer los datos, y almacenando únicamente un índice de tripletas fichero-clave-sesión. Cada ensayo se lee bajo demanda durante el entrenamiento, lo que permite trabajar con las 45 sesiones sin que quepan en memoria.

Un detalle relevante para la capa de sesión: **el índice de sesión es global**, es decir, corresponde a la posición del día en la lista completa de sesiones y no en el subconjunto empleado. De este modo, cada día dispone de su propia matriz aunque sólo aparezca en validación, lo que es imprescindible para el experimento de generalización a días no vistos.

Se emplean las 45 sesiones disponibles, incluidas las cuatro que sólo aportan datos de entrenamiento por carecer de partición de validación. La partición de prueba oficial no se utiliza, al no distribuirse con etiquetas: todas las cifras reportadas corresponden a la partición de validación.

## 6.3. Receta de entrenamiento

Todos los experimentos comparten una receta base, sobre la que cada experimento modifica únicamente los parámetros bajo estudio. La cuadro 6.1 recoge sus valores.

**Tabla 6.1:** Receta de entrenamiento de referencia.

| Parámetro              | Valor            | Comentario                                               |
|------------------------|------------------|----------------------------------------------------------|
| Sesiones               | 45               | Todas las disponibles                                    |
| Épocas                 | 30               | 70 en el experimento final                               |
| Optimizador            | AdamW            | Necesario para que el decaimiento regularice             |
| Decaimiento de pesos   | $10^{-2}$        | Valor estándar de AdamW                                  |
| Tasa de aprendizaje    | $10^{-3}$        | Componentes nuevos                                       |
| Escala del codificador | 0,05             | Tasa reducida para pesos preentrenados                   |
| Calentamiento          | 1.500 pasos      |                                                          |
| Recorte de gradiente   | 100              | Véase el comentario más abajo                            |
| Tamaño de lote         | 16 (ordenado)    | Con acumulación de 2                                     |
| Precisión mixta        | Activada         |                                                          |
| Adaptador              | Profundo         | $d = 512$ , 2 bloques residuales, submuestreo $\times 2$ |
| Capa de sesión         | Activada         |                                                          |
| Objetivo CTC           | Carácter         | 29 clases                                                |
| Peso de CTC            | 1,0              | Decodificador desactivado                                |
| InterCTC               | 0,3 en la capa 2 |                                                          |
| Normalización          | Ninguna          | Los datos ya vienen normalizados por bloque              |
| Aumento                | Activado         | Ruido 0,5, desplazamiento 0,2                            |

Tres decisiones de esta receta proceden de errores detectados y merecen justificación explícita:

**Recorte de gradiente.** Con un umbral de 5 y normas de gradiente del orden de 200, el registro mostraba que **el cien por cien de los pasos se recortaba**, descartando un factor de unas cuarenta veces la magnitud del gradiente en cada actualización. El recorte había dejado de ser una salvaguarda frente a valores atípicos para convertirse en una reescala sistemática de la tasa de aprendizaje efectiva.

**Optimizador.** Con Adam clásico, el decaimiento de pesos se suma al gradiente y queda reescalado por el denominador adaptativo, de modo que apenas regulariza. Cualquier experimento sobre regularización exige AdamW.

**Búsqueda de algoritmos de convolución.** Con lotes ordenados por longitud, cada lote tiene una forma distinta y la búsqueda automática del mejor algoritmo con-

volucional se repite para cada forma nueva, lo que en la primera época supone más de un centenar de búsquedas exhaustivas. Se desactiva.

## 6.4. Selección de modelo y parada temprana

Esta sección documenta el error metodológico de mayor impacto detectado durante el trabajo, y su corrección.

ESPnet emplea **dos criterios distintos e independientes**: uno para decidir qué punto de control se conserva como mejor modelo, y otro para decidir cuándo detener el entrenamiento por falta de mejora. El segundo tiene como valor por omisión la pérdida de validación, con independencia de lo que se haya configurado para el primero.

Las consecuencias fueron dos, ambas conforme a lo previsto teóricamente en la apartado 4.4:

1. Con un peso de CTC reducido, la pérdida total está dominada por el término de atención, que toca fondo en torno a la cuarta época gracias al conocimiento previo del inglés del modelo preentrenado, mientras la tasa de error de la tarea sigue descendiendo. Seleccionar por pérdida conservaba modelos claramente peores.
2. En régimen de sobreajuste, la pérdida de validación asciende mientras la tasa de error continúa mejorando. Con el criterio por omisión, un experimento se detuvo en la época 38 de 60 teniendo su mejor tasa de error en la 37 y una pendiente aún descendente: se interrumpió un entrenamiento que seguía mejorando.

La corrección consiste en fijar explícitamente ambos criterios sobre la tasa de error de la rama CTC, y en retrasar el inicio del cómputo de paciencia hasta la época 15, de modo que la fase inicial de calentamiento, en la que las variaciones son erráticas, no dispare la parada.

Este episodio motiva un principio adoptado en el resto del trabajo: **ningún valor por omisión de la herramienta se da por bueno sin comprobarlo**, y en particular ninguno que gobierne la selección de modelo.

## 6.5. Protocolo de evaluación

### 6.5.1. Dos regímenes de decodificación

Se emplean dos procedimientos de evaluación con propósitos distintos:

**CTC voraz.** Un paso hacia delante y el máximo por instante, con el colapso estándar. No pasa por el decodificador, de modo que **mide exclusivamente lo que el codificador extrae de la señal neuronal** y no puede beneficiarse del conocimiento previo del inglés del modelo preentrenado. Es barato, por lo que se ejecuta sobre la partición de validación completa.

**Búsqueda en haz sobre la salida CTC.** Decodificación por haz sobre las distribuciones de la rama CTC, integrando el modelo de lenguaje de  $n$ -gramas según el procedimiento de la apartado 6.8. Mide **el sistema completo**, codificador más reconstrucción lingüística. Es mucho más costosa por ensayo, por lo que se limita a 200 ensayos.

Conviene precisar que, en la configuración de referencia, el peso de la rama CTC es la unidad y por tanto **el decodificador autorregresivo está desactivado** (apartado 5.2). No existe, en consecuencia, ninguna decodificación generativa: toda la evaluación se realiza sobre la salida de la rama CTC. Los experimentos iniciales del trabajo, anteriores a la fijación de la receta, sí operaron con peso de CTC inferior a la unidad y decodificador activo; sus cifras no son comparables con las reportadas aquí y se señalan como tales cuando aparecen.

Ambos procedimientos seleccionan sus ensayos **de forma equiespaciada** a lo largo de la partición, de modo que el subconjunto evaluado por la búsqueda en haz es una muestra representativa del que evalúa la decodificación voraz y no un tramo sesgado de él. Inicialmente la búsqueda en haz tomaba simplemente los primeros ensayos mientras la decodificación voraz ya usaba una muestra repartida; con 45 sesiones ordenadas cronológicamente, los primeros ensayos corresponden todos a los primeros días de registro, de modo que ambas cifras se estaban midiendo sobre poblaciones distintas y no eran comparables entre sí.

### 6.5.2. Dos definiciones de tasa de error por carácter

La decodificación voraz reporta dos cifras que no son intercambiables:

- Una tasa **micro**, calculada como el cociente entre la suma de errores y la suma de longitudes de referencia sobre todos los ensayos, exactamente como la calcula ESPnet internamente. Es la única comparable con la columna del registro de entrenamiento y, por tanto, la que permite comparar experimentos entre sí.
- Una tasa **por ensayo**, promediada sobre ensayos tras normalizar el texto. Se aproxima más a la noción intuitiva de tasa de error, pero los ensayos cortos que fallan la penalizan de forma desproporcionada.

Las tablas de resultados indican explícitamente cuál de las dos se reporta en cada caso.

### 6.5.3. Precisión de la medida

Toda diferencia entre configuraciones debe contrastarse contra el ruido de muestreo. El procedimiento de evaluación calcula el error estándar de la media y su intervalo de confianza al 95 % sobre la muestra de ensayos evaluados.

Los órdenes de magnitud son los siguientes: con 200 ensayos, el error estándar de la tasa media es de aproximadamente 0,02, lo que da un intervalo de confianza de entre  $\pm 0,03$  y  $\pm 0,05$ ; con la partición de validación completa, unos 1.400 ensayos con las 45 sesiones, el intervalo baja a alrededor de  $\pm 0,015$ .

La implicación es directa y condiciona la lectura de los resultados: **dos configuraciones cuyas tasas difieran menos que la suma de sus intervalos de confianza no son distinguibles con esa muestra**. Diferencias del orden de un punto porcentual, que en iteraciones tempranas del trabajo se interpretaron como mejoras, están por debajo del umbral de resolución de la medida. Por este motivo la evaluación voraz se realiza sobre la validación completa siempre que es posible.

### 6.5.4. Comprobaciones de sanidad

Un modelo que ignora por completo la entrada produce siempre la misma hipótesis, típicamente una frase genérica procedente del conocimiento previo del decodificador. La evaluación detecta y advierte de este caso comparando las hipótesis entre ensayos, lo que evita interpretar como resultado lo que es un fallo de conexión entre la señal y el modelo.

## 6.6. Principios metodológicos

### 6.6.1. Una variable por experimento

Cada experimento modifica un único parámetro respecto a la receta base. Este principio se adoptó tras un error documentado: un experimento sobre aumento de datos modificó simultáneamente la activación del aumento y el valor del descarte aleatorio, de modo que el resultado no permitía atribuir el efecto a ninguno de los dos y hubo que descartarlo.



### 6.6.2. Presupuesto de cómputo igualado

Las comparaciones entre configuraciones asignan a todas las variantes el mismo número de épocas y desactivan la parada temprana. Esto hace la comparación justa, pero implica que **las cifras de las ablaciones no corresponden al rendimiento máximo alcanzable** por cada configuración, sino a su rendimiento bajo presupuesto común. Sólo el experimento final se entrena hasta convergencia.

Este criterio obliga además a una cautela al interpretar resultados negativos: una configuración que aún no ha alcanzado su meseta al final del presupuesto no puede declararse peor sin extenderla. Cuando la pendiente de la curva en la última época indica que el descenso continúa, el experimento se repite con un presupuesto mayor antes de extraer conclusiones.

### 6.6.3. Trazabilidad

Cada experimento se identifica mediante una huella derivada de su configuración **completa**, no de la etiqueta. Esto permite tres cosas: saltar experimentos ya ejecutados, reintentar automáticamente los que terminaron en error por agotamiento de memoria o interrupción, y garantizar que dos filas de la tabla con la misma etiqueta corresponden efectivamente a la misma configuración. La versión inicial derivaba la huella sólo de un subconjunto de parámetros, lo que en un barrido puede hacer que un experimento nuevo se identifique erróneamente como ya realizado.

De cada ejecución se conservan la fila de resultados, el registro completo de entrenamiento y la curva de métricas por época.

## 6.7. Diseño experimental

Los experimentos se organizan en bloques, cada uno de los cuales responde a una pregunta concreta. La cuadro 6.2 los resume.

**Tabla 6.2:** Bloques experimentales y pregunta a la que responde cada uno.

| Bloque | Objetivo  | Pregunta                                                                                   |
|--------|-----------|--------------------------------------------------------------------------------------------|
| A      | OE3 (PI1) | ¿Aporta algo el preentrenamiento frente a la inicialización aleatoria?                     |
| D      | OE6 (PI1) | ¿Cómo varía el rendimiento con la cantidad de codificador preentrenado que se conserva?    |
| H      | OE9       | ¿Es reproducible el hallazgo, y se le ha dado al modelo preentrenado su mejor oportunidad? |
| F      | OE6       | Si los pesos no importan, ¿importa el tamaño de la arquitectura?                           |
| B      | OE4 (PI2) | ¿Qué granularidad lingüística de salida es la adecuada?                                    |
| E      | OE7       | ¿Cuánto se degrada el sistema en días no vistos?                                           |
| C      | OE5       | ¿Qué capacidad necesita el módulo adaptador?                                               |
| G      | OE9       | Configuración final entrenada hasta convergencia.                                          |
| LM     | OE8 (PI3) | ¿Reordena el modelo de lenguaje las hipótesis o guía la búsqueda?                          |

### 6.7.1. Bloque A: el control científico

Compara tres condiciones con receta idéntica, siendo la inicialización del codificador la única variable: pesos preentrenados con ajuste completo, pesos preentrenados con codificador congelado, e inicialización aleatoria con ajuste completo. Es el bloque del que depende la primera pregunta de investigación y, en consecuencia, el primero que se ejecuta.

### 6.7.2. Bloque D: la curva de conservación

Extiende el bloque anterior con puntos intermedios: descongelado de las dos y de las cuatro capas superiores del codificador, y adaptación de rango bajo sobre codificador congelado. Convierte tres puntos aislados en una curva que relaciona la cantidad de codificador preentrenado conservado con el rendimiento. Incluye además la extensión del experimento con codificador congelado a un presupuesto mayor, por aplicación del criterio de cautela descrito anteriormente.

### 6.7.3. Bloque H: blindaje del hallazgo

Anticipa las dos objeciones evidentes a un resultado que contradice la literatura previa. La primera es la reproducibilidad: el experimento se repite con dos semillas adicionales, ya que un hallazgo de este calibre no puede sostenerse sobre una única ejecución. La segunda es la equidad del ajuste: la tasa de aprendizaje reducida para el codificador se eligió para no destruir los pesos preentrenados, de modo que se ejecuta una variante con una tasa más alta antes de concluir que el preentrenamiento no aporta.

### 6.7.4. Bloque F: el tamaño de la arquitectura

Reduce el número de bloques del codificador, algo posible únicamente con inicialización aleatoria. Responde a la pregunta que sigue de forma natural al bloque A y aporta el eje de coste-eficiencia del trabajo.

### 6.7.5. Bloque B: granularidad de salida

Compara carácter, fonema, subpalabra propia, palabra de vocabulario cerrado y subpalabra preentrenada. Conforme a lo establecido en la apartado 4.7, la comparación se cierra sobre el error por palabra del texto decodificado, único eje común entre unidades distintas.

### 6.7.6. Bloque E: generalización a días no vistos

Aparta por completo las cinco últimas sesiones, que no participan en el entrenamiento y constituyen el conjunto de evaluación. Se comparan tres tratamientos del día no visto: dejar su matriz de sesión en la identidad, sustituirla por la media de las matrices entrenadas, y prescindir por completo de la capa de sesión.

### 6.7.7. Bloque C: capacidad del adaptador

Compara las variantes lineal y convolucional unidimensional frente a la profunda. Es el bloque de menor prioridad y el primer candidato a recortarse si el presupuesto se agota.

Este bloque exige una precaución que lo distingue del resto. Las variantes lineal y convolucional unidimensional aplican un factor de submuestreo fijo de cuatro, mientras que la receta de referencia emplea la variante profunda con factor dos. Comparadas directamente, la arquitectura del adaptador quedaría **confundida con la resolución temporal**, que es exactamente el defecto experimental que este trabajo se propone

evitar. La comparación debe ejecutarse, por tanto, igualando el factor de submuestreo en todas las variantes, o bien reportando explícitamente ambos ejes y absteniéndose de atribuir la diferencia a la arquitectura.

### 6.7.8. Bloque G: configuración final

Entrena la mejor configuración resultante con un presupuesto ampliado y parada temprana activa, para determinar hasta dónde llega realmente el sistema.

## 6.8. Protocolo de decodificación con modelo de lenguaje

### 6.8.1. Construcción del modelo de lenguaje

El modelo de  $n$ -gramas se estima **exclusivamente sobre las transcripciones de la partición de entrenamiento**. Emplear las de validación constituiría una fuga de información que inflaría artificialmente el resultado.

La estimación se realiza con suavizado de Kneser-Ney mediante las herramientas de KenLM [heafield2011kenlm]. Esta elección corrige un problema metodológico de la primera implementación: al no disponer inicialmente del estimador, el fichero de modelo se generaba con suavizado aditivo en Python puro, que **penaliza aproximadamente el doble a un trigramma disperso que a un bigrama**. El resultado previo de que el trigramma rendía peor que el bigrama mezclaba, por tanto, el efecto del orden del modelo con el del suavizado, y no era interpretable. Con Kneser-Ney la comparación queda limpia, confirme o no la conclusión anterior.

### 6.8.2. Ajuste de los hiperparámetros de fusión

Los coeficientes de la ecuación (4.9) —el peso del modelo de lenguaje y la bonificación por inserción de palabra— se ajustan mediante barrido en rejilla sobre la partición de validación, y la rejilla completa se conserva para poder documentar la sensibilidad del sistema a ambos.

### 6.8.3. Cifras reportadas

Se reportan por separado, y esto es deliberado:

- La tasa de error por carácter con decodificación voraz, que **mide el codificador**.

- La tasa de error por palabra con búsqueda en haz y modelo de lenguaje, que mide **el sistema completo**.
- El límite *oracle* de la lista  $n$ -mejor acústica, definido en la apartado 4.6.4, que permite discriminar si el modelo de lenguaje reordena o guía la búsqueda.

Mezclar las dos primeras en una única cifra ocultaría cuánto del rendimiento procede de la lectura de la señal y cuánto de la reconstrucción lingüística posterior, que es precisamente una de las preguntas que el trabajo pretende responder.

# Capítulo 7

## Resultados y análisis

### 7.1. Resultados cuantitativos

**Tabla 7.1:** Resultados por bloque experimental.

| Experimento             | WER ↓ | CER ↓ | PER ↓ |
|-------------------------|-------|-------|-------|
| A1 (init. aleatoria)    | —     | —     | —     |
| A2 (encoder congelado)  | —     | —     | —     |
| v3 (preentrenado, ref.) | —     | —     | —     |

### 7.2. Análisis de errores

### 7.3. Comportamiento por sesión

### 7.4. Curvas de entrenamiento

### 7.5. Discusión

## Capítulo 8

### Limitaciones y trabajo futuro

8.1. Limitaciones computacionales

8.2. Limitaciones del enfoque

8.3. Líneas de trabajo futuro

## Capítulo 9

## Conclusiones



## Bibliografía (stub de prueba)

## Apéndice A

### Código del pipeline de entrenamiento

## Apéndice B

### Configuraciones de ESPnet-EZ

## Apéndice C

### Análisis exploratorio adicional